

Figures from the Versioning & Identity Research

Companion plate-book to the 114-page deep reading
24 sources · 116 figures & page-renders · 2026-06-11

PDF papers are shown as page-renders of pages bearing figure/table captions (the figure with its caption in context). HTML sources are shown as their extracted inline figures. Eight sources have no extractable figures and are noted with the reason.

Angle 1 — Program-Unit Identity

The Big Idea (Unison documentation) · 2024

<https://www.unison-lang.org/docs/the-big-idea/>

⚠ Figures are CSS/inline on the page; none embedded as extractable images.

On Product Versioning for Hypertexts · 2005

<https://users.soe.ucsc.edu/~ejw/scml2/papers/Nguyen-Chapter9.pdf>

7 PAGE RENDERS

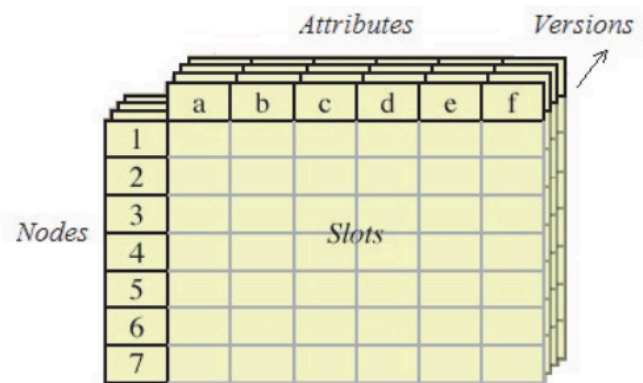


Fig. 3. Data model

A *constant slot* is immutable; such a slot can only be given a value once, when it is defined. A *simple slot* may be assigned even after it has been defined. The third kind of slot is the *versioned slot*, which may have different values in different versions (*slot revisions*). A *sequence* is a container with slots of the same data type. It has a unique identifier. Sequences may be fixed or variable in size and share common slots together. Once we add versioning, the attribute table gets a third dimension: the version.

4.2 Product versioning

The version dimension follows the product versioning model. Instead of focusing on individual components, Molhado versions a software project as a whole (see Figure 4). All system objects including (atomic and composite) components and hypertext structures are versioned in a *uniform, global version space*. Object properties can be defined as versioned or un-versioned. A *version* is global across the whole project and is a point in a *tree-structured discrete time* abstraction, rather than being a *particular state* of an object as in total and composition versioning models. That is, the third dimension in the attribute table in Figure 3 is tree-structured and versions move discretely from one point to another.

The state of the whole software system is captured at certain discrete time points and only these captured versions can be retrieved in later sessions (for example, the versions *v1.0*, *v2.0*, *v3.0*, etc in Figure 4). The *current version* is the version designating the current state of the project. When the current version is set to a captured version, the state of the whole project is set back to that version (for example, *v2.0* in Figure 4). Changes made to versioned objects of the project at the current version create a temporary version, *branching off* the current version. That temporary version will only be recorded if a user explicitly requests that it be captured. To record the history of an individual object, the whole project is captured. Capturing the whole project is quite efficient because the versioning system only records changes and works at a very small granularity [3].

network. Figure 6c) shows part of the attribute table for the network at version $v1$ (document nodes in the attribute table are not shown except $n(\text{section1})$). In this example, attribute “content” defines for each document node a slot that contains a string value. The “ref” cell for an anchor node (e.g. node 1) contains a reference to the corresponding document node (e.g. $n(\text{section1})$). Figure 6f) shows the attribute table at version $v2$. Node 5 was deleted since *doc3* was removed. Node 3 now has only one child. Node 9 (representing *link3*) and node 10 (representing *class3*) are just created. The “ref” cell for node 10 points to document node representing *class3*. The “content” slot of $n(\text{section1})$ has changed from “old” to “new” to reflect the change in the content of (section 1, doc 1) in the network.

In Molhado’s product versioning model, a version is a point in tree-structured discrete time line. Therefore, returning to previous state of entire project is a basic versioning operation. When users set the current version to a recorded version, the state of entire software project will be set back to that version. Thus, documents as well as linkbases and hypertext networks will regain their contents and structures at that version.

4.6 Version control for links

Via the example in Figure 6, we can see that the directed graph versioning scheme also takes care of recording the history of individual hyperlinks (represented as a graph node). This is because the connection structure of a graph node and associated slots are captured over time.

Let us revisit the first issue with version control for hyperlinks (see Figure 1). In this case, Molhado avoids this problem. The nature of product versioning in Molhado allows developers to navigate to the right destination at the current version. In Molhado (see Figure 7), two global versions $v1$ and $v2$ would be created. At the version $v1$, the connection would be “ $A \rightarrow \text{the link} \rightarrow B$ ”, and at the version $v2$, the connection would be “ $A \rightarrow \text{the link} \rightarrow C$ ”. When the current version is selected explicitly or implicitly, the destination of the link will be automatically chosen at the current version.

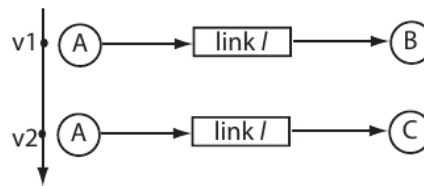


Fig. 7. No problem with link versioning in Molhado

Let us consider the second issue with versioning for hyperlinks (see Figure 2). This scenario can be handled by putting all hypermedia entities including links and resources in one global product versioning space such as in Molhado. Thus, when users select $v2$ the current version, the destination of the link l would

be the new version of the resource *B* (see Figure 8). The key idea is that, in Molhado, link traversal occurs only among nodes at the *current* version.

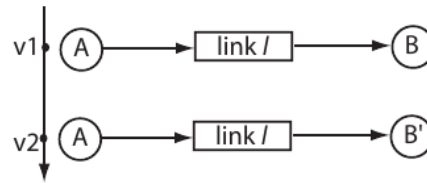


Fig. 8. No problem with link versioning in Molhado (2)

The third issue with link versioning is whether a system should treat a hyperlink as a first-class object or a relation of anchor values. In Molhado, a hyperlink is a first-class entity. It is represented as a node in a directed graph (representing a hypertext network). Each graph node might be associated with multiple attributes (representing properties of hypermedia entities). As explained, via the fine-grained versioning scheme for attributed directed graphs, a hypertext network as well as an individual link is versioned (see the example in Figure 6). In general, when a versioned attribute value associated with the hyperlink changes, or the connection of member anchors changes, or a document node on which a member anchor rests changes, a new version must be created. In this case, a new global version point is created in the tree-structured discrete time line, rather than a new version of resources or a new version of links.

In brief, Molhado provides configuration management for hypermedia structure as well as version control for individual links and document nodes without having problems that occurred in existing versioned hypermedia systems. Now, let us revisit other issues mentioned by Østerbye and relate them to Molhado.

4.7 Revisiting other issues

Immutability of versions: There are two characteristics in Molhado’s product versioning model that suggested us a simple solution for this issue. The first one is that Molhado can control which aspects of a software project are versioned or un-versioned. The other one is that resource versions and hyperlink structure versions are maintained in the same global version space. To address this issue, in Molhado, users are allowed to specify the set of “substantial” links (i.e. the ones that if modified would create a new version), while all other links would be considered as annotation or comment links and would not require a new version if changed. Substantial links are modeled as *versioned slots* defined by “href” attribute, while other links as *simple slots* defined by an additional attribute. Therefore, when a substantial link is created and linked to a resource at the current version, a temporary version will be created and this change is hold at this temporary version. But when a non-substantial link is created, the change is hold at the current version and no temporary version will be created.

network, select a hypertext network to be *active*, to import and export a hypertext network from and to XLink format at any version. Services for links include link creation, deletion, renaming, attribute's value viewing, and link history viewing. Services for anchors include deleting an anchor, adding an anchor into the active link, removing an anchor off some link, renaming an anchor, and displaying the structural unit that the anchor refers to. More details on this set of library functions can be found in another document [32].

5.2 User operations

This set of library functions can be used in any editing or development environment. Automatic link generation algorithms could be employed to create links and our infrastructure could be used to store versions of documents and links. Molhado's hypertext versioning services were integrated into the Software Concordance (SC) development environment [31] (see Figure 9). Configuration management tasks are handled by Molhado's configuration management infrastructure [33]. The rest of this section describes the distinguished versioned hypermedia functionality in the SC environment.

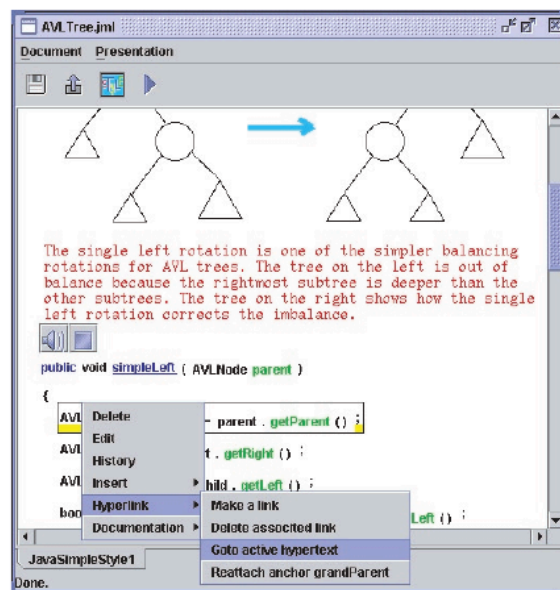


Fig. 9. Structured document editor

Molhado and the SC user interfaces support a variety of transactions. First of all, a user can open an existing project. After selecting the *current (working) version* from a project history window, the system displays the project's directory structure and documents in a project structure window. From this

window, the user can choose to edit, delete, import, or export any document and hypertext network. Also, via this window, the user can graphically modify the project's directory structure. If any modification is made to the versioned components at the current version, a new version would be temporarily created, branching off the current version. Subsequent modifications will not change the temporary status of the version until a *capture* or a *commit* command is issued. If the user does not want to keep the temporary version, he/she can *discard* it. Otherwise, the user can *capture* the state of the project at a version. The *capture* command changes a temporary version into a captured one. A unique name as well as date, authors, and descriptions can be attached to the newly captured version for later retrieval. A captured version plays the role of a checkpoint that the user can retrieve and refer to.

The user can *branch off* a version by just *switching* the current version and then starting the modifications. While working on one version, the user can always *switch* to work on (view or modify) any other version. This switching feature allows the user to work on many versions at the same time during one session. This capability is called *multi-version editing*. The user may *commit* changes at any time. Upon issuing this command, the user is asked which *uncaptured, temporary versions* should be saved and the chosen versions are then saved along with any already captured versions. Only the differences are stored. Saving complete version snapshots can improve version access time.

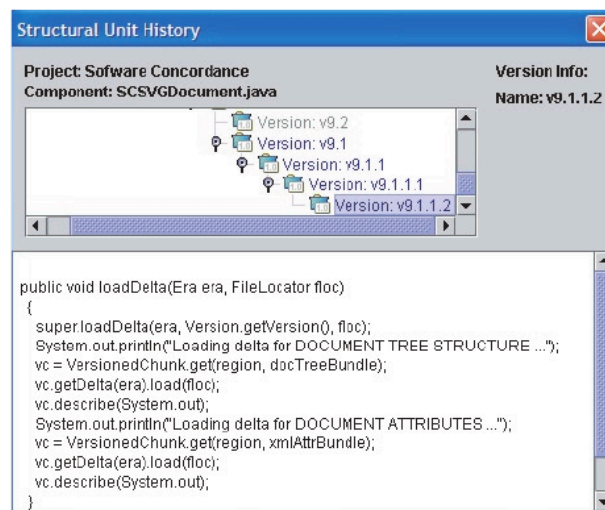


Fig. 10. History of a document node

The user can open a software document at the current version (e.g. XML, HTML, Java program, SVG graphic, UML diagram). An appropriate editor will be invoked. The editors are enhanced by versioned hypermedia services. Figure 9 shows the structured editor for a Java program. When the user right-clicks

on a document node, a popup menu is displayed to allow the user to create an anchor at the node and add it to the active hypertext network, to open the active hypertext network of an anchor defined at the document node, or to create or delete a HTML-style hyperlink at the document node.

In addition, the user is able to view a document node's history (see Figure 10). Note that the method "loadDelta" was not created until *v9.1*. Therefore, the earlier versions on the top window are "disabled". If the selected logical unit is the root of a document, the history of entire document will be displayed.

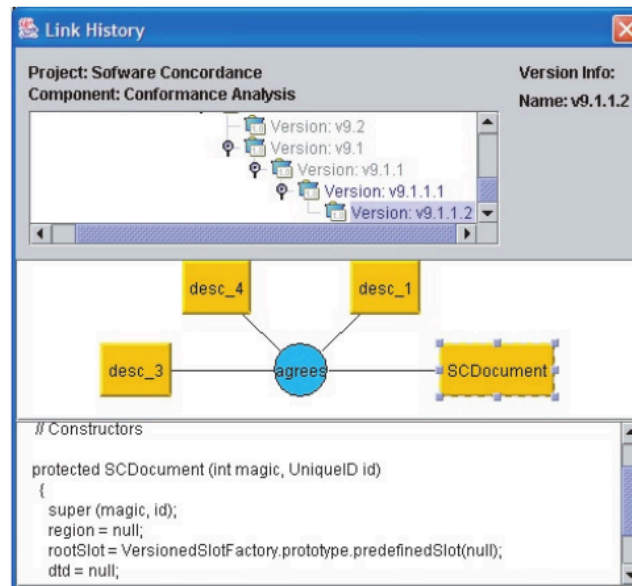


Fig. 11. History of a hyperlink

At any time, the user can create a hyperlink and make it active in a hypertext network. Then, he/she can add anchors into that hyperlink or any other hyperlinks currently managed in the system. Members of the hyperlink and its properties can be modified and changes are recorded. Figure 11 shows the history of the link "agrees" between several requirement/design items and the class "SC-Document". The constructor of the class was displayed in the bottom window since the user clicked on the corresponding anchor.

The user can also display, modify, or navigate through a hypertext network via a simple editor. In Figure 12, there are two types of links: causal and non-causal. There are directed edges coming from source anchors to a causal link, and from a causal link to its target anchors. There are only non-directed edges for non-causal links (e.g. link "n1"). While non-causal links are directly represented by our hyperlinks, causal links are extended from them with additional attributes. Via anchors, the user can traverse to corresponding document nodes.

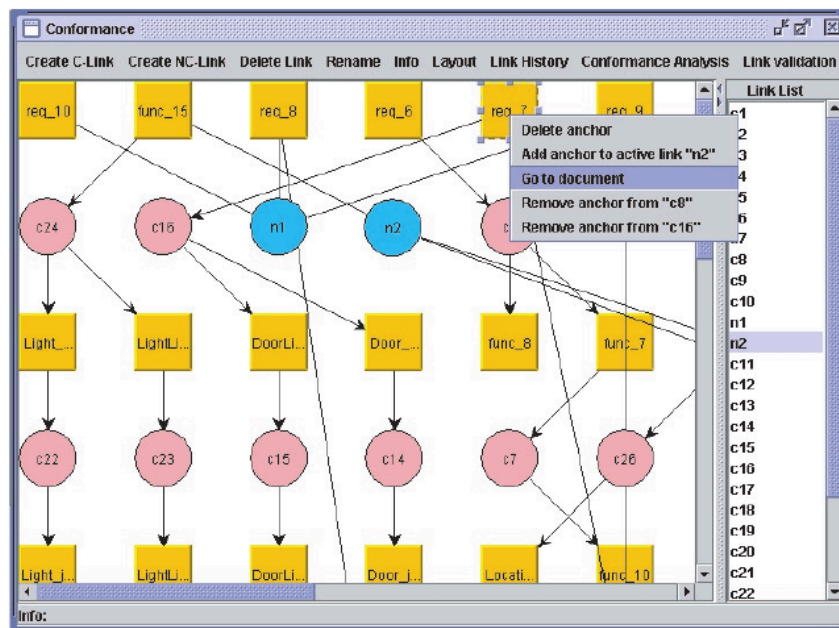


Fig. 12. A hypertext network

In this environment, multiple hypertext networks can also be defined during a software development process. With our tool, document contents will not be changed when a new network is defined since hypertext structure is separated from document content.

6 Issues for future research

While using product versioning, we have encountered three problems that have not been described for composition or total versioning systems. We plan to explore these issues in the near future.

6.1 Hyperlinks across different versions

Molhado's current interfaces are not sufficient to allow users to easily create hyperlinks between different versions. This is because links only exist within a single version. So, there is no way to say "make a link to document D at version v_k ," because the linking interfaces have no parameter for the version. While this may not be the most common use case for versioned hypermedia, it is certainly necessary to support it and we have identified two approaches that should suffice.

The first solution is to allow links to other versions to be made as *computational links*. The anchors of a computational link are essentially queries against a database of links. Using the example mentioned above, the anchor's query would

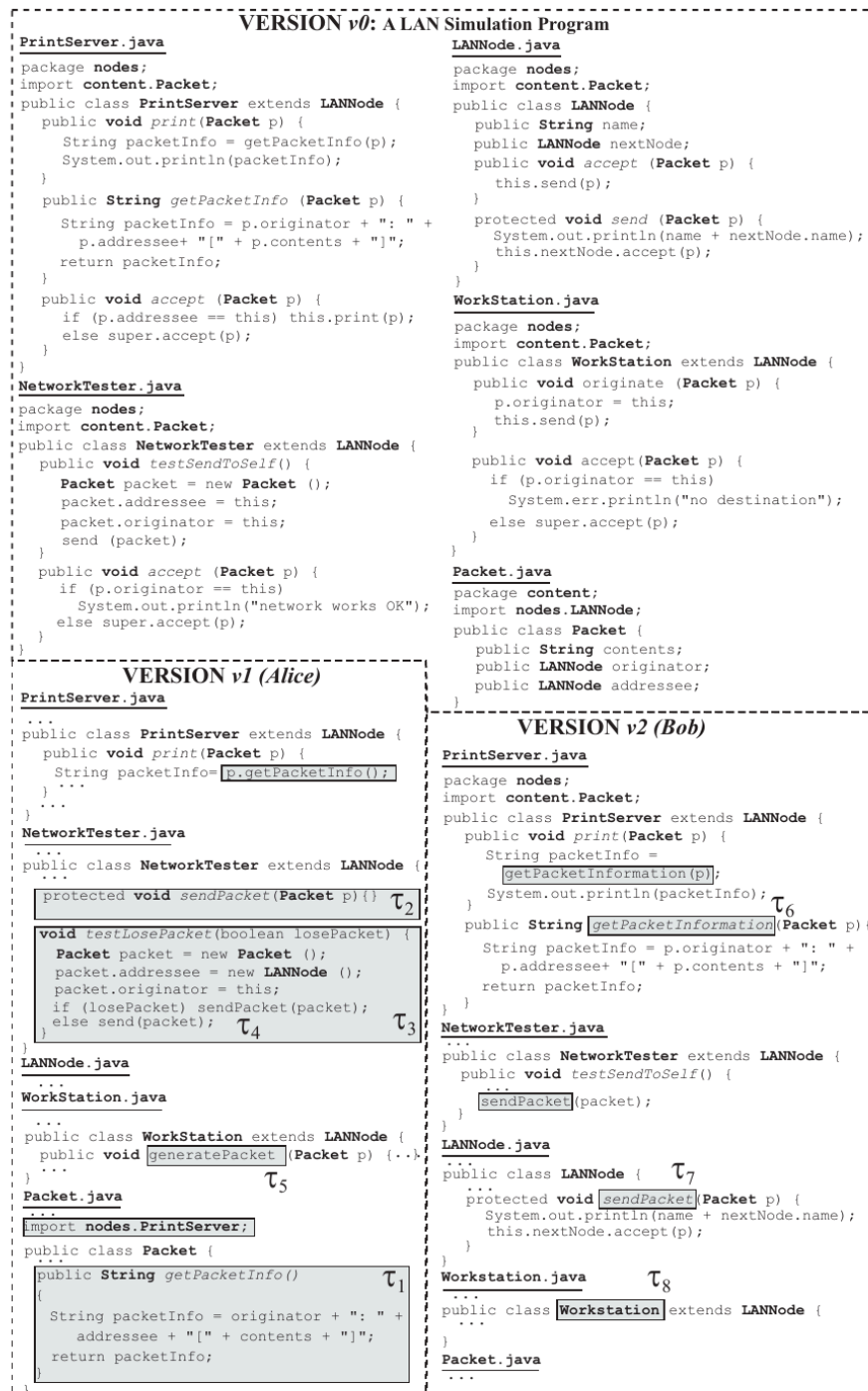


Figure 1. Motivating Example. Boxes show the changes in each version.

lists installed packages and the right panes shows known repositories. Various operations may be performed via the button pane and the menus of the two list panes.

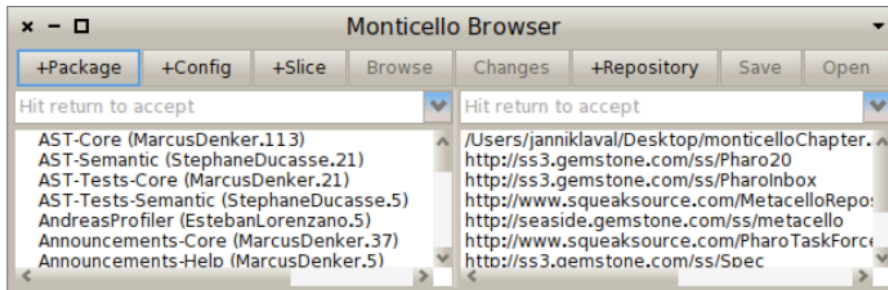


Figure 7.1: The Monticello Browser.

Creating a package

Monticello manages versions of *packages*. A package is essentially a named set of classes and methods. In fact, a package is an object—an instance of `PackageInfo`—that knows how to identify the classes and methods that belong to it.

We would like to version our `PerfectTest` class. The right way to do this is to define a package—called `Perfect`—containing `PerfectTest` and all the related classes and methods we will introduce later. For the moment, no such package exists. We only have a *category* called (not coincidentally) `Perfect`. This is perfect, since Monticello will map categories to packages for us.

 Press the `+Package` in the Monticello browser and enter `Perfect`.

Voilà! You have just created the *Perfect* Monticello package.

Monticello packages follow a number of important naming conventions for class and method categories. Our new package named *Perfect* contains:

- All classes in the category *Perfect*, or in categories whose names start with *Perfect-*. For now this includes only our `PerfectTest` class.
- All methods belonging to *any* class (in any category) that are defined in a protocol named **perfect* or **Perfect*, or in protocols whose names start with **perfect-* or **Perfect-*. Such methods are known as *extensions*. We don't have any yet, but we will define some very soon.
- All methods belonging to any classes in the category *Perfect*, or in categories whose names begin with *Perfect-*, *except* those in protocols whose

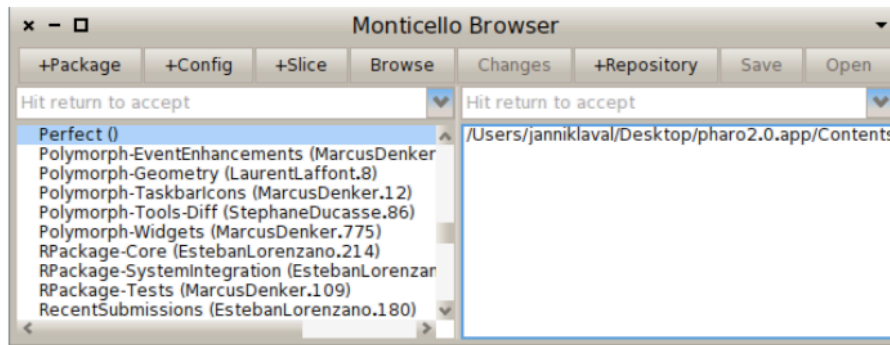


Figure 7.2: Creating the Perfect package.

names start with * (*i.e.*, those belonging to *other* packages). This includes our `testPerfect` method, since it belongs to the protocol running.

Committing changes

Note in Figure 7.2 that the `Save` button is disabled (greyed out).

Before we save our `Perfect` package, we need to specify *where* we want to save it. A *repository* is a package container, which may either be local to your machine or remote (accessed over the network). Various protocols may be used to establish a connection between your Pharo image and a repository. As we will see later (Section 7.5), Monticello supports a large choice of repositories, though the most commonly used is HTTP, since this is the one used by SmalltalkHub.

At least one repository, called `package-cache`, is set up by default, and is shown as the first entry in the list of repositories on the right-hand side of your Monticello browser (see Figure 7.1). The `package-cache` is created automatically in the local directory where your Pharo image is located. It will contain a copy of all the packages you download from remote repositories. By default, copies of your packages are also saved in the `package-cache` when you save them to a remote server.

Each package knows which repositories it can be saved to. To add a new repository to the selected package, press the `+Repository` button. This will offer a number of choices of different kinds of repository, including HTTP. For the rest of the chapter we will work with the `package-cache` repository, as this is all we need to explore the features of Monticello.

 Select the directory repository named `package cache`, press `Save`, enter an appropriate log message, and `Accept` to save the changes.

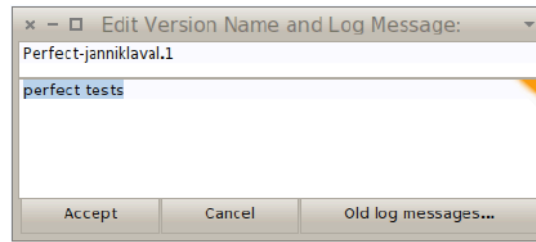


Figure 7.3: You may set a new version name and a commit message when you save a version of a package.

The Perfect package is now saved in `package-cache`, which is nothing more than a directory contained in the same directory as your Pharo image. Note, however, that if you use any other kind of repository (e.g., HTTP, FTP, another local directory), a copy of your package will also be saved in the `package-cache`.

🔗 Use your favorite file browser (e.g., Windows Explorer, Finder or XTerm) to confirm that a file `Perfect-XX.1.mcz` was created in your package cache. `XX` corresponds to your name or initials.⁸

A *version* is an immutable snapshot of a package that has been written to a repository. Each version has a unique version number to identify it in a repository. Be aware, however, that this number is *not* globally unique—in another repository you might have the same file identifier for a *different snapshot*. For example, `Perfect-onierstrasz.1.mcz` in another repository might be the *final*, deployed version of our project! When saving a version into a repository, the next available number is automatically assigned to the version, but you can change this number if you wish. Note that version branches do not interfere with the numbering scheme (as with CVS or Subversion). As we shall see later, versions are by default ordered by their version number when viewing a repository.

Class extensions

Let's implement the methods that will make our tests green.

🔗 Define the following two methods in the class `Integer`, and put each method in a protocol called `*perfect`. Also add the new boundary tests. Check that the tests are now green.

⁸In the past, the convention was for developers to log their changes using only their initials. Now, with many developers sharing identical initials, the convention is to use an identifier based on the full name, such as "apblack" or "AndrewBlack".

```

Integer»isPerfect
  ^ self > 1 and: [self divisors sum = self]

Integer»divisors
  ^ (1 to: self - 1 ) select: [ :each | (self rem: each) = 0 ]

PerfectTest»testPerfectBoundary
  self assert: 0 isPerfect not.
  self assert: 1 isPerfect not.

```

Although the methods on `Integer` do not belong to the *Perfect* category, they *do* belong to the *Perfect* package since they are in a protocol whose name starts with `*` and matches the package name. Such methods are known as *class extensions*, since they extend existing classes. These methods will be available *only* to someone who loads the *Perfect* package.

“Clean” and “Dirty” packages

Modifying the code in a package with any of the development tools makes that package *dirty*. This means that the version of the package in the image is different from the version that has been saved or loaded.

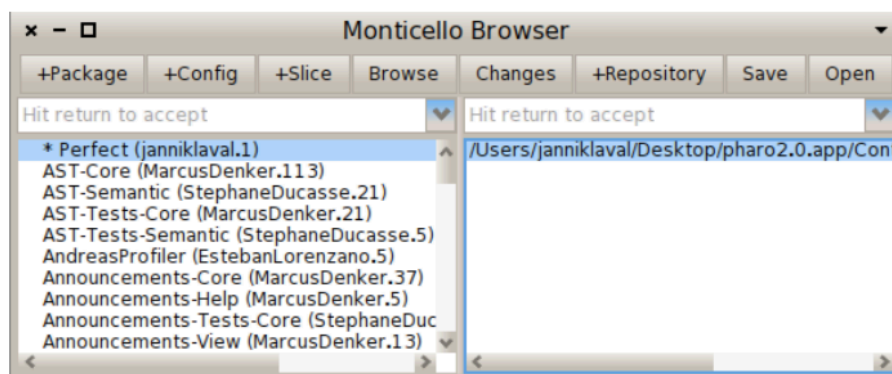


Figure 7.4: Modifying our *Perfect* package will “dirty” it.

In the Monticello browser, a dirty package can be recognized by an asterisk (*) preceding its name. This indicates which packages have uncommitted changes, and therefore need to be saved into a repository if those changes are not to be lost. Saving a dirty package cleans it.

 Try the **Browse** and **Changes** buttons to see what they do. **Save** the changes to the *Perfect* package. Confirm that the package is now “clean” again.

The Repository inspector

The contents of a repository can be explored using a repository inspector, which is launched using the `Open` button of Monticello (cf Figure 7.5).

 Select the package–cache repository and open it. You should see something like Figure 7.5.

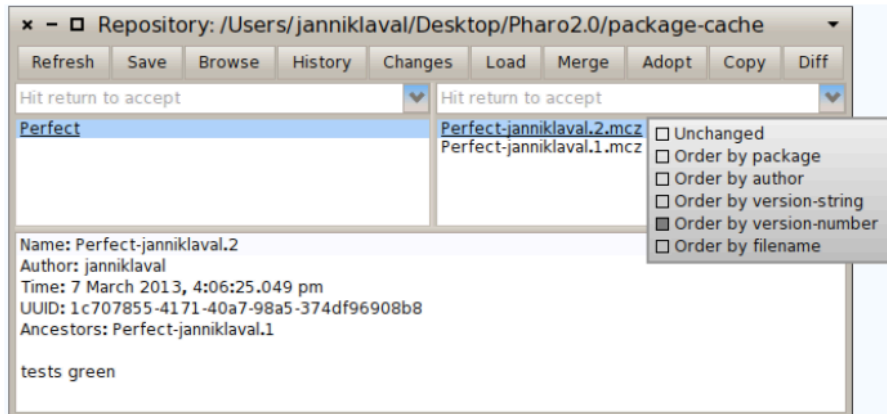


Figure 7.5: A repository inspector.

All the packages in the repository are listed on the left-hand side of the inspector:

- an underlined package name means that this package is installed in the image;
- a **bold underlined** name means that the package is installed, but that there is a more recent version in the repository;
- a name in a normal typeface means that the package is not installed in the image.

Once a package is selected, the right-hand pane lists the versions of the selected package:

- an underlined version name means that this version is installed in the image;
- a **bold** version name means that this version is not an ancestor of the installed version. This may mean that it is a newer version, or that it belongs to a different branch from the installed version;

- a version name displayed with a normal typeface shows an older version than the installed current one.

Action-clicking the right-hand side of the inspector opens a menu with different sorting options. The `unchanged` entry in the menu discards any particular sorting. It uses the order given by the repository.

Loading, unloading and updating packages

At present we have two versions of the Perfect package stored safely in our package-cache repository. We will now see how to unload this package, load an earlier version, and finally update it.

 Select the Perfect package and its repository in the Monticello browser. Action-click on the package name and select `unload package`.

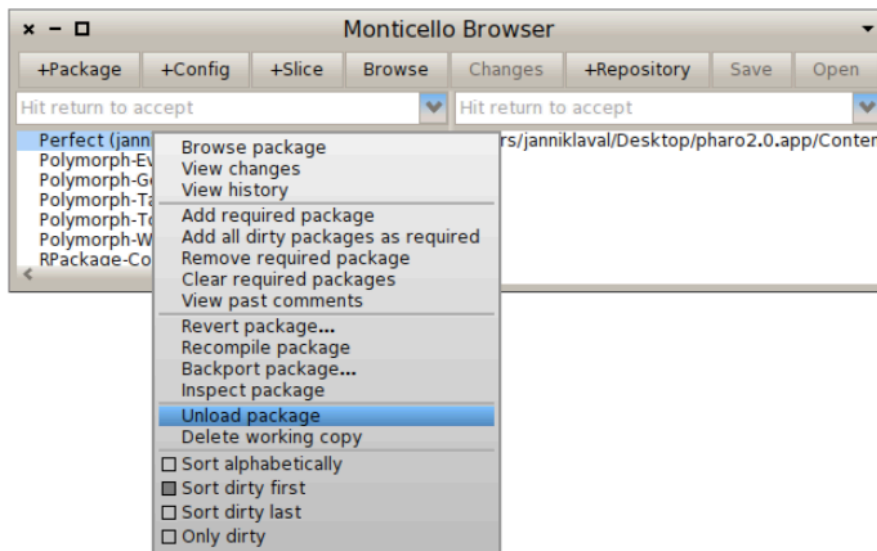


Figure 7.6: Unloading a package.

You should now be able to confirm that the Perfect package has vanished from your image!

 In the Monticello browser, select the package-cache in the repository pane, without selecting anything in the package pane, and `Open` the repository inspector. Scroll down and select the Perfect package. It should be displayed in a normal typeface, indicated that it is not installed. Now select version 1 of the package and `Load` it.

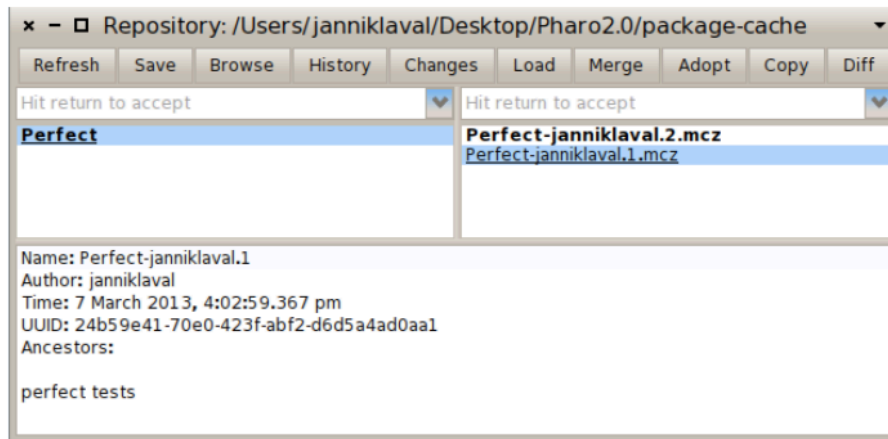


Figure 7.7: Loading an earlier version.

You should now be able to verify that only the original (red) tests are loaded.

 Select the second version of the **Perfect** package in the repository inspector and **Load** it. You have now updated the package to the latest version.

Now the tests should be green again.

Branching

A *branch* is a line of development versions that exists independently of another line, yet still shares a common ancestor version if you look far enough back in time.

You may create new version branch when saving your package. Branching is useful when you want to have a new parallel development. For example, suppose your job is to maintain a software in your company. One day a different division asks you for the same software, but with a few parts tweaked for them, since they do things slightly differently. The way to deal with this situation is to create a second branch of your program that incorporate the tweaks, while leaving the first branch unmodified.

 From the repository inspector, select version 1 of the **Perfect** package and **Load** it. Version 2 should again be displayed in bold, indicating that it no longer loaded (since it is not an ancestor of version 1). Now implement the following two Integer methods and place them in the `*perfect` protocol, and also modify the existing `PerfectTest` test method as follows:

```

Integer>isPerfect
  self < 2 ifTrue: [ ^ false ].
  ^ self divisors sum = self

Integer>divisors
  ^ (1 to: self - 1 ) select: [ :each | (self \\ each) = 0]

PerfectTest>testPerfect
  self assert: 2 isPerfect not.
  self assert: 6 isPerfect.
  self assert: 7 isPerfect not.
  self assert: 28 isPerfect.

```

Once again the tests should be green, though our implementation of perfect numbers is slightly different.

🕒 *Attempt to load version 2 of the Perfect package.*

Now you should get a warning that you have unsaved changes.

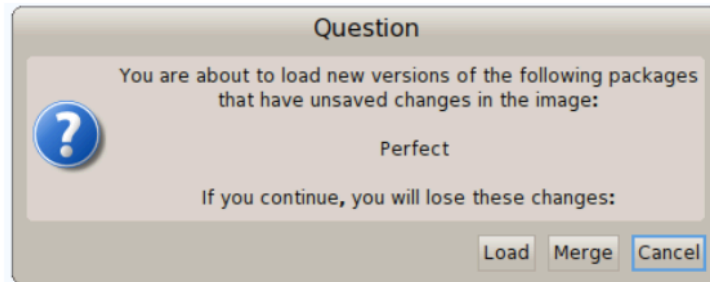


Figure 7.8: Unsaved changes warning.

🕒 *Select **Cancel** to avoid overwriting your new methods. Now **Save** your changes. Enter your log message, and **Accept** the new version.*

Congratulations! You have now created a new branch of the Perfect package.

🕒 *If you still have the repository inspector open, **Refresh** it to see the new version (Figure 7.9).*

Merging

You can merge one version of a package with another using the **Merge** button in the Monticello browser. Typically you will want to do this when (i)

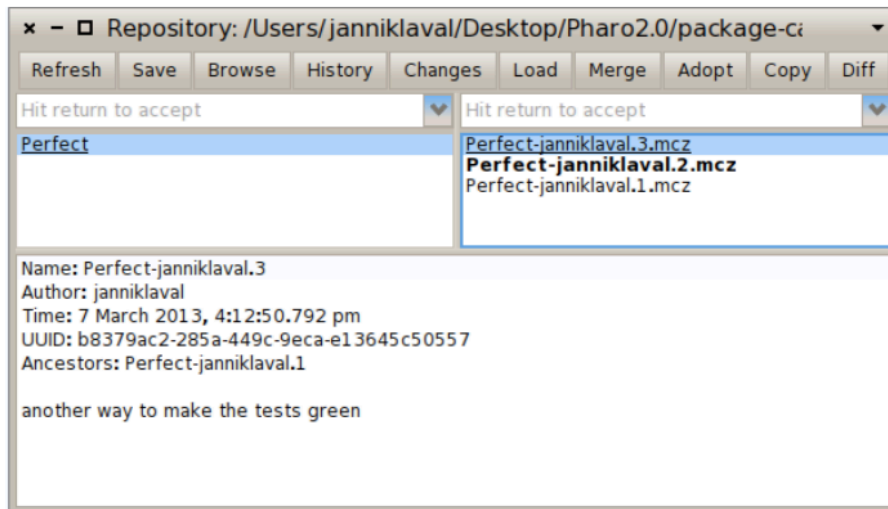


Figure 7.9: Versions 2 and 3 are separate branches of version 1.

you discover that you have been working on a out-of-date version, or (ii) branches that were previously independent have to be re-integrated. Both scenarios are common when multiple developers are working on the same package.

Consider the current situation with our Perfect package, as illustrated at the left of Figure 7.10. We have published a new version 3 that is based on version 1. Since version 2 is also based on version 1, versions 2 and 3 constitute independent branches.

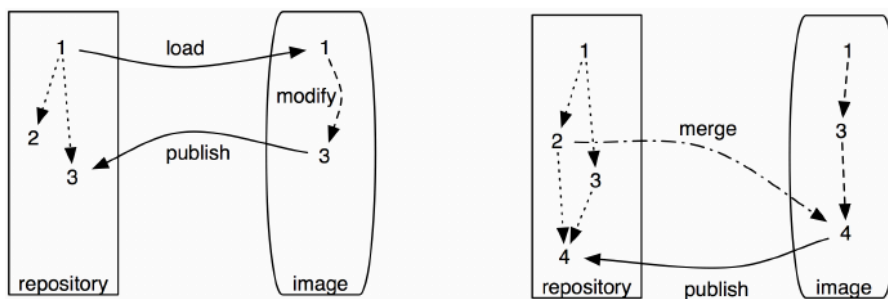


Figure 7.10: Branching (left) and merging (right).

At this point we realize that there are changes in version 2 that we would like to merge with our changes from version 3. Since we have version 3 currently loaded, we would like to merge in changes from version 2, and publish a new, merged version 4, as illustrated at the right of Figure 7.10.

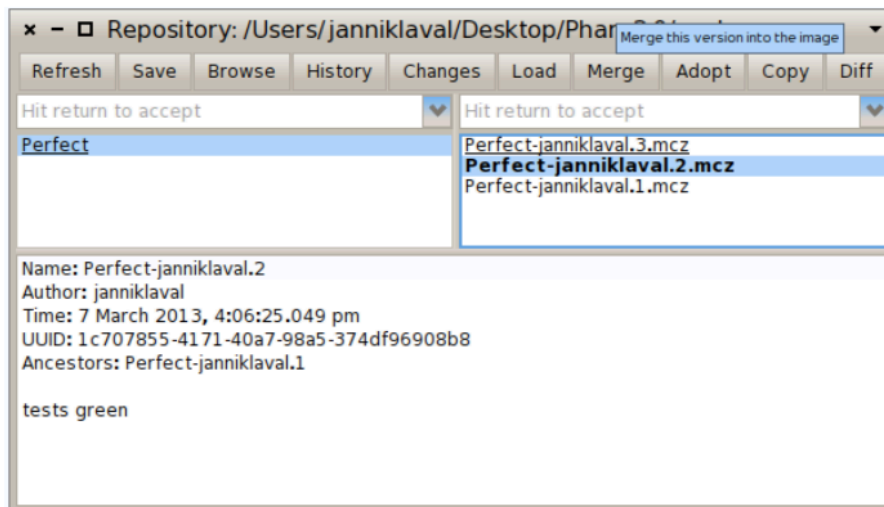


Figure 7.11: Select a separate branch (in bold) to be merged.

 Select version 2 in the repository browser, as shown in Figure 7.11, and click the **Merge** button.

The merge tool is a tool that allows for fine-grained package version merging. Elements contained in the package to-be-merged are listed in the upper text pane. The lower text pane shows the definition of a selected element.

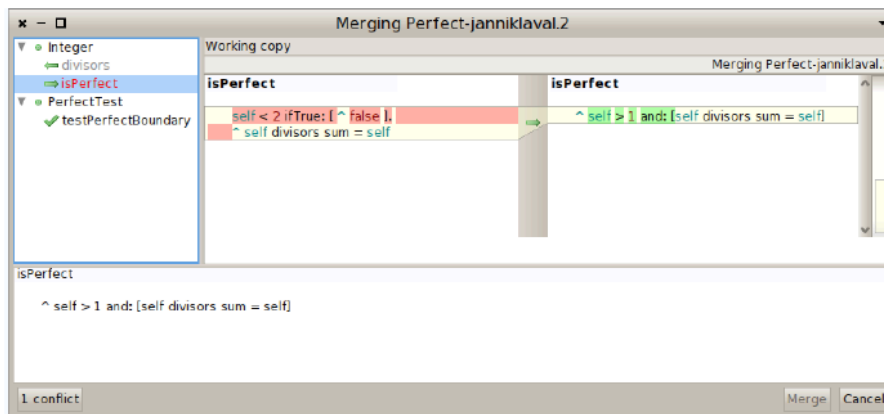


Figure 7.12: Version 2 of the Perfect package being merged with the current version 3.

In Figure 7.12 we see the three differences between versions 2 and 3 of

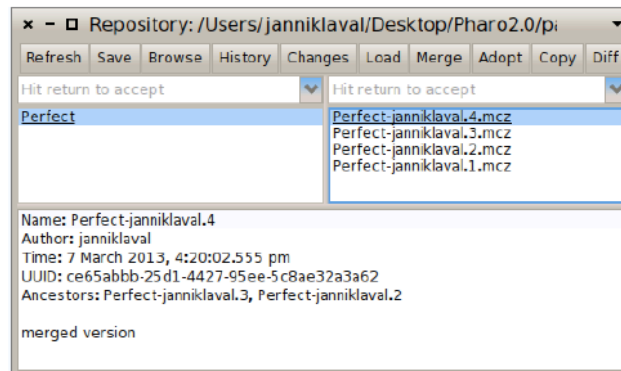


Figure 7.13: All older versions are now ancestors of merged version 4.

Browse

The **Browse** button opens a “snapshot browser” to display the contents of a package. The advantage of the snapshot browser over the browser is its ability to display class extensions.

 Select the **Perfect** package and click the **Browse** button.

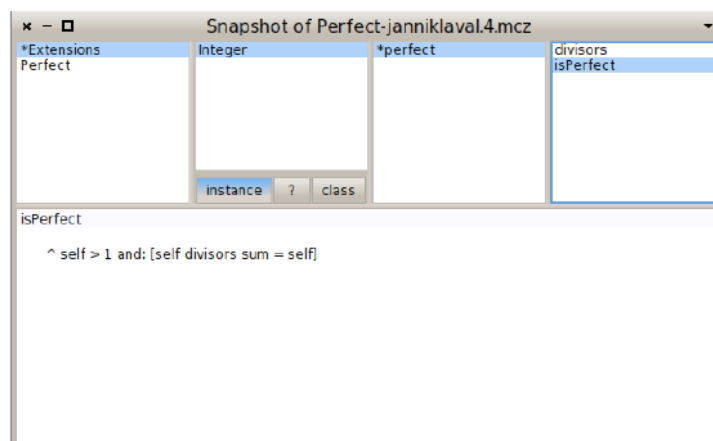


Figure 7.14: The snapshot browser reveals that the **Perfect** package extends the class **Integer** with 2 methods.

For example, Figure 7.14 shows the class extensions defined in the *Perfect* package. Note that code cannot be edited here, though by action-clicking, if your environment has been set up accordingly) on a class or a method name you can open a regular browser.

It is a good practice to always browse the code of your package before publishing it, to ensure that it really contains what you think it does.

Changes

The `Changes` button computes the difference between the code in the image and the most recent version of the package in the repository.

 *Make the following changes to PerfectTest, and then click the `Changes` button in the Monticello browser.*

```
PerfectTest»testPerfect
self assert: 2 isPerfect not.
self assert: 6 isPerfect.
self assert: 7 isPerfect not.
self assert: 496 isPerfect.

PerfectTest»testPerfectTo1000
self assert: ((1 to: 1000) select: [:each | each isPerfect]) = #(6 28 496)
```

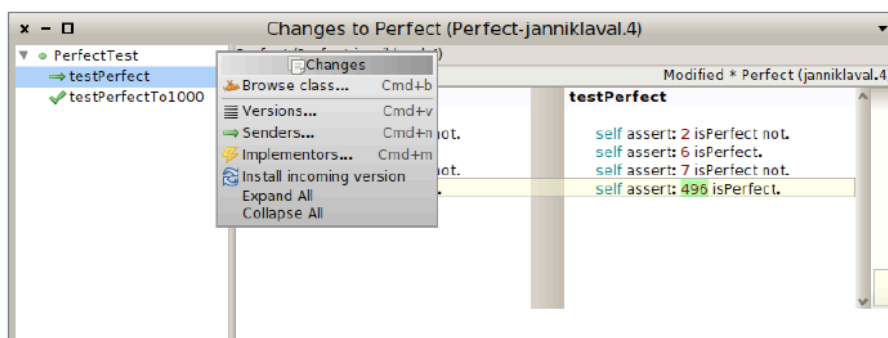


Figure 7.15: The patch browser shows the difference between the code in the image and the most recently committed version.

Figure 7.15 shows that the Perfect package has been locally modified with one changed method and one new method. As usual, action-clicking on a change offers you a choice of contextual operations.

7.3 Advanced topics

Now we will have a look at several advanced topics, including history, managing dependencies, making configuration, and class initialization.

History

By action-clicking on a package, you can select the item **History**. It opens a version history viewer that displays the comments committed along with each version of the selected package (see Figure 7.16). The versions of the package, in this case *Perfect*, are listed on the left, while information about the selected version is displayed on the right.

 Select the *Perfect* package, right click and select the **History** item.

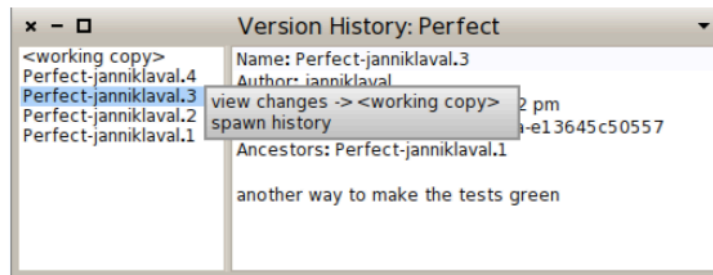


Figure 7.16: The version history viewer provides information about the various versions of a package.

By action-clicking on a particular version, you can explore the changes with respect to the current working copy of the package loaded in the image, or spawn a new history browser relative to the selected version.

Dependencies

Most applications cannot live on their own and typically require the presence of other packages in order to work properly. For example, let us have a look at Pier⁹, a meta-described content management system. Pier is a large piece of software with many facets (tools, documentations, blog, catch strategies, security, ...). Each facet is implemented by a separate package. Most Pier packages cannot be used in isolation since they refer to methods and classes defined in other packages. Monticello provides a dependency mechanism for declaring the *required packages* of a given package to ensure that it will be correctly loaded.

Essentially, the dependency mechanism ensures that all required packages of a package are loaded before the package is loaded itself. Since required packages may themselves require other packages, the process is applied recursively to a tree of dependencies, ensuring that the leaves of the tree are loaded before any branches that depend on them. Whenever new

⁹<http://source.lukas-renggli.ch/pier>

versions of required packages are checked in, then new versions of the packages that depend on them will automatically depend on the new versions.

Dependencies cannot be expressed across repositories. All requiring and required packages must live in the same repository.

Figure 7.17 illustrates how this works in Pier. Package Pier-All is an *empty package* that acts as a kind of umbrella. It requires Pier-Blog, Pier-Caching and all the other Pier packages.

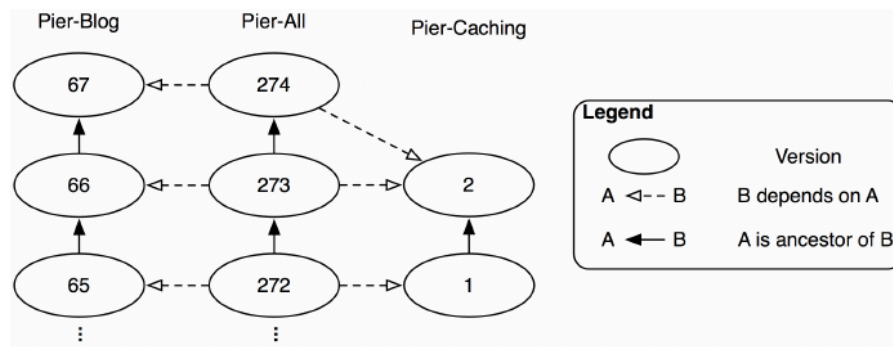


Figure 7.17: Dependencies in Pier.

Because of these dependencies, installing Pier-All causes all the other Pier packages to be installed. Furthermore, when developing, the only package that needs to be saved is Pier-All; all dependent dirty packages are saved automatically.

Let us see how this works in practice. Our Perfect package currently bundles the tests together with the implementation. Suppose we would like instead to separate these into separate packages, so that the implementation can be loaded without the tests. By default, however, we would like to load everything.

 Take the following steps:

- Load version 4 of the Perfect package from the package cache
- Create a new package in the browser called NewPerfect-Tests and drag the class PerfectTest to this package
- Rename the *perfect protocol of the Integer class to *newperfect-extensions (action-click to rename it)

- In the Monticello browser, add the packages `NewPerfect-All` and `NewPerfect-Extensions`.
- Add `NewPerfect-Extensions` and `NewPerfect-Tests` as required packages to `NewPerfect-All` (action-click on `NewPerfect-All`)
- Save package `NewPerfect-All` in the package-cache repository. Note that Monticello prompts for comments to save the required packages too.
- Check that all three packages have been saved in the package cache.
- Monticello thinks that `Perfect` is still loaded. Unload it and then load `NewPerfect-All` from the repository inspector. This will cause `NewPerfect-Extensions` and `NewPerfect-Tests` to be loaded as well as required packages.
- Check that all tests run.

Note that when `NewPerfect-All` is selected in the Monticello browser, the dependent packages are displayed in bold (see Figure 7.18).

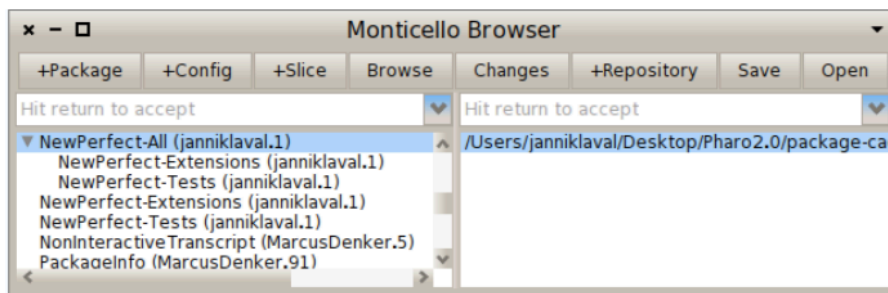


Figure 7.18: `NewPerfect-All` requires `NewPerfect-Extensions` and `NewPerfect-Tests`.

If you further develop the `Perfect` package, you should only load or save `NewPerfect-All`, not its required packages.

Here is the reason why:

- If you load `NewPerfect-All` from a repository (package-cache, or anywhere else), this will cause `NewPerfect-Extensions` and `NewPerfect-Tests` to be loaded from the same repository.
- If you modify the `PerfectTest` class, this will cause the `NewPerfect-Tests` and `NewPerfect-All` packages to both become dirty (but not `NewPerfect-Extensions`).

```
(path asFileReference).
MCRepositoryGroup default addRepository: repo ].
```

Using SmalltalkHub

SmalltalkHub is a online repository that you can use to store your Monticello packages. An instance is running and accessible from <http://smalltalkhub.com/>.

Welcome to SmalltalkHub

The free, opensource, Smalltalk projects management application

[No account yet? Register!](#)

WARNING The following is a preview of the exploration features of SmalltalkHub. More to come!

323 repositories, **365** users registered and **25599** packages uploaded.

 **Recently registered users**

-  bprior (bprior)
-  Dellany (Slavik Skorokhid)
-  leobm (leobm)
-  Arctorion (Arctorion)
-  johnsmith (johnsmith)
-  saykirtt (saykirtt)
-  AubAurelAntho (AubAurelAntho)
-  MarionSertARien (MarionSertARien)
-  didseb (didseb)
-  Nono (Nono)
-  fdodino (fdodino)
-  vnqls (vnqls)
-  FabrizioPerin (FabrizioPerin)
-  mva (mva)

 **Recently created projects**

- [Phratch](#) (created the Thu Mar 07 2013)
- [Wonderland](#) (created the Wed Mar 06 2013)
- [Units](#) (created the Wed Mar 06 2013)
- [Leds](#) (created the Wed Mar 06 2013)
- [LED](#) (created the Wed Mar 06 2013)
- [SebJim](#) (created the Wed Mar 06 2013)
- [MyProject](#) (created the Wed Mar 06 2013)
- [led](#) (created the Wed Mar 06 2013)
- [Ledcard](#) (created the Wed Mar 06 2013)
- [Phexample](#) (created the Tue Mar 05 2013)
- [PublicIssueTracker](#) (created the Tue Mar 05 2013)
- [NameExperience](#) (created the Mon Mar 04 2013)
- [LibLLVM](#) (created the Mon Mar 04 2013)
- [LightsOutGame](#) (created the Mon Mar 04 2013)
- [Painter](#) (created the Sun Mar 03 2013)

Figure 7.19: SmalltalkHub, the online Monticello code repository.

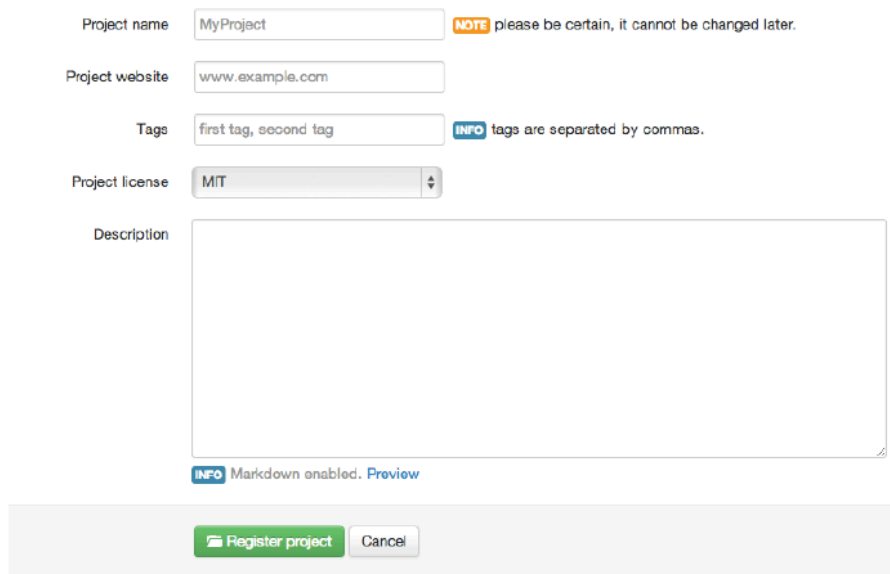
 Use a web browser to visit the main page <http://smalltalkhub.com/>. When you select a project, you should see this kind of repository expression:

```
MCHttpRepository
location: 'http://smalltalkhub.com/mc/PharoExtras/Phexample/main'
user: ''
password: ''
```

Add this repository to Monticello by clicking **+Repository**, and then selecting **HTTP**. Fill out the template with the URL corresponding to the project — you can copy the above repository expression from the web page and paste it into the template. Since you are not going to commit new versions of this package, you do not need to fill in the user and password. **Open** the repository, select the latest version of Phexample and click **Load**.

Pressing the **Join** link on the SmalltalkHub home page will probably be your first step if you do not have a SmalltalkHub account. Once you are a member, **+ New Project** allows you to create a new project.

Create a new project



Project name **NOTE** please be certain, it cannot be changed later.

Project website

Tags **INFO** tags are separated by commas.

Project license

Description

INFO Markdown enabled. [Preview](#)

Figure 7.20: Repositories under SmalltalkHub are configurable.

SmalltalkHub offers options (cf. Figure 7.20) to configure a project repository: tags may be assigned, a license may be chosen with access for people who are not part of the project may be restricted (private, public), and users may be defined to be members of the project. You also can create a team that shares projects.

7.6 The .mcz file format

Versions are stored in repositories as binary files. These files are commonly call “mcz files” as they carry the extension .mcz. This stands for “Monticello

Angle 2 — Version-Model Theory

Version Models for Software Configuration Management · 1998

<https://dl.acm.org/doi/10.1145/280277.280280>

⚠ No open-access copy found (gated at ACM). The AND/OR-graph figures are unavailable.

Making Data Structures Persistent · 1989

<https://dl.acm.org/citation.cfm?id=64317>

15 PAGE RENDERS

embedded in it. We navigate through the persistent structure as follows. When an ephemeral access step applied to version i accesses field f of a node, we access the value in the corresponding fat node whose field name is f , choosing among several such values the one with maximum version stamp no greater than i .

We also need an auxiliary data structure to store the access pointers of the various versions. This structure consists of an array of pointers for each access pointer name. After update operation i , we store the current values of the access pointers in the i th positions of these access arrays. With this structure, initiating access into any version takes $O(1)$ time.

Remark. The only purpose of nodes having version stamps is to make sure that each node only contains one value per field name per version. These stamps are not needed if there is some other mechanism for keeping track during update operation i of the newest field values of nodes created during update i . In order to navigate through the structure it suffices to regard each original field value in a node as having a version stamp of zero.

Figure 1 shows a persistent binary search tree constructed using the fat node method. The update operations are insertions and deletions, performed as described in Section 1. Version stamps on nodes are easy to dispense with in this application. Since the original pointers in each node are known to be null, they too can be omitted. (The version f pointer field of a node is taken to be null if no field f pointer stored in the node has a version stamp less than or equal to i .)

The fat node method applies to any linked data structure and uses only $O(1)$ space per ephemeral update step in the worst case, but it has two drawbacks. First, the fat nodes must be represented by linked collections of fixed-size nodes. This poses no fundamental difficulty but complicates the implementation. Second, choos-

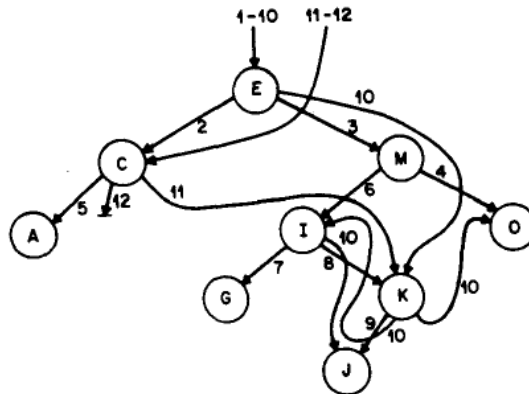


FIG. 1. A partially persistent search tree built using the fat node method, for the sequence of update operations consisting of insertions of E , C , M , O , A , I , G , K , J , followed by deletion of M , E , and A . The "extra" pointers are labeled with their version stamps. Left pointers leave the left sides of nodes and right pointers leave the right sides. The version stamps and original null pointers of nodes are omitted, since they are unnecessary.

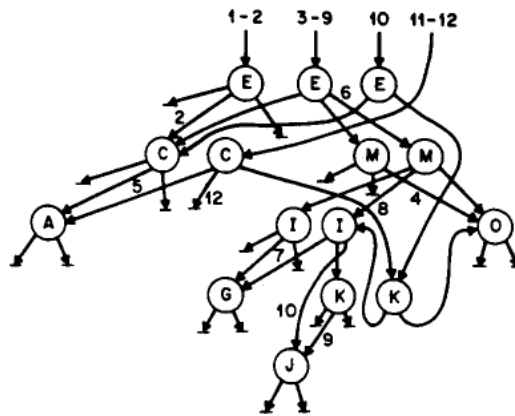


FIG. 2. A partially persistent search tree built using the node-copying method, for the same sequence of update operations as in Fig. 1. The version stamps of nodes and the copy pointers are omitted, since they are unnecessary.

2.3.4. Space and Time Analysis

It remains for us to analyze the space needed for the persistent structure and the time needed for update steps. We shall derive an amortized $O(1)$ bound on the space and time per update step using the "potential" technique [33]. To obtain this bound, it suffices to choose e to be any integer constant such that $e \geq p$.

Recall that a persistent node is live if it has not been copied and dead otherwise. All updates to fields are in live nodes, except for erasures of inverse pointers, which can occur in dead nodes. We define the *potential* of the persistent structure to be $e/(e-p+1)$ times the number of live nodes it contains minus $1/(e-p+1)$ times the number of unused extra pointer fields in live nodes. Observe that the potential of the initial (empty) structure is zero and that the potential is always nonnegative, since any node has at most e unused extra pointer fields. We define the *amortized space cost* of an update operation to be the number of nodes it creates plus the net increase in potential it causes. With this definition, the total number of nodes created by a sequence of update operations equals the total amortized space cost minus the net increase in potential over the sequence. Since the initial potential is zero and the final potential is nonnegative, the net potential increase over any sequence is nonnegative, which implies that the total amortized space cost is an upper bound on the total number of nodes created.

The key part of the analysis is to show that the amortized space cost of an update operation is linear in the number of update steps. Consider an update operation that performs u update steps. Each update step has an amortized space cost of $O(1)$ and adds at most one node to S . Consider the effect of the postprocessing that empties S . Let $t \leq u$ be the number of nodes in S at the beginning of the postprocessing and let k be the number of nodes copied during the postprocessing. Each time a node is copied during postprocessing, a live node with potential

this difficulty, we impose a total ordering on the versions consistent with the partial ordering defined by the version tree. We represent this total ordering by a list of the versions in the appropriate order. We call this the *version list*. When a new version, say i , is created, we insert i in the version list immediately after its parent (in the version tree). The resulting list defines a preorder on the version tree, as can easily be proved by induction. This implies that the version list has the following crucial property: for any version i , the descendants of i in the version tree occur consecutively in the version list, starting with i (see Fig. 3). We shall refer to the direction toward the front of the version list (from a given item) as *leftward* and the direction toward the back of the list as *rightward*.

In addition to performing insertions in the version list, we need to be able to determine, given two versions i and j , whether i precedes or follows j in the version list. This *list order problem* has been addressed by Dietz [10], by Tsakalidis [3, 4], and most recently by Dietz and Sleator [11]. Dietz and Sleator [11] proposed a list representation supporting order queries in $O(1)$ worst-case time, with an $O(1)$ amortized time bound for insertion. They also proposed a more complicated representation in which both the query time and the insertion time are $O(1)$ in the worst case.

3.2. The Fat Node Method

Having dealt in a preliminary way with the navigation issue, let us investigate how to use fat nodes to make a linked structure fully persistent. We use the same node organization as in Section 2; namely, each fat node contains the same fields as

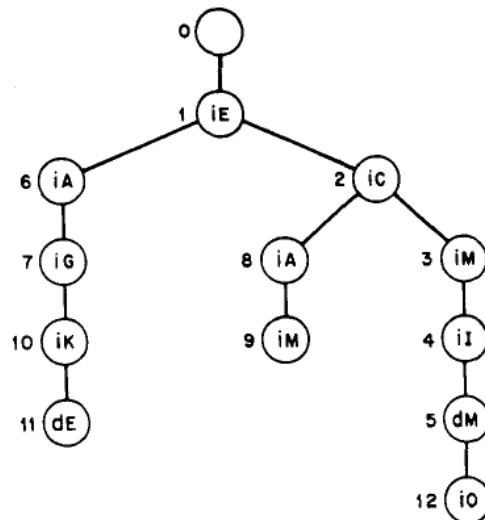


FIG. 3. A version tree. Each node represents an update operation; an "i" or "d" indicates an insertion or deletion of the specified item. The nodes are labeled with the indices of the corresponding operations. The version list is 1, 6, 7, 10, 11, 2, 8, 9, 3, 4, 5, 12.

node method can be easily established using a proof by induction on the number of update steps to show that the proper correspondence between the ephemeral structure and the persistent structure is maintained.

Figure 4 shows a fully persistent binary search tree built using the fat node method. In this application, we can as in the partially persistent case omit the version stamps of nodes and the original null pointers of nodes.

The fat node method provides full persistence with the same asymptotic efficiency as it provides partial persistence; namely, the worst-case space cost per update step is $O(1)$ and the worst-case time cost per access and update step is $O(\log m)$, provided that each set of field values in a fat node is stored in a search tree, ordered by version stamp. A more accurate bound is $O(\log h)$ time per access or update step, where h is the maximum number of changes made to an ephemeral node. As in the case of partial persistence, the fat node method applies even if the in-degree of ephemeral nodes is not bounded by a constant.

3.3. The Node-Splitting Method

We improve over the fat node method by using a variant of node copying. We shall call the variant *node splitting*, since it resembles the node splitting used to perform insertions in *B*-trees [1]. The major difference between node splitting and node copying is that in the former, when a node overflows, a new copy is created and roughly half the extra pointers are moved from the old copy to the new one, thereby leaving space in both the old and new copies for later updates. The efficient implementation of node splitting is more complicated than that of node copying, primarily because the correct maintenance of inverse pointers requires care. Also, access pointers for old versions must sometimes be changed. This requires the maintenance of inverse pointers for access pointers as well as for node-to-node pointers.

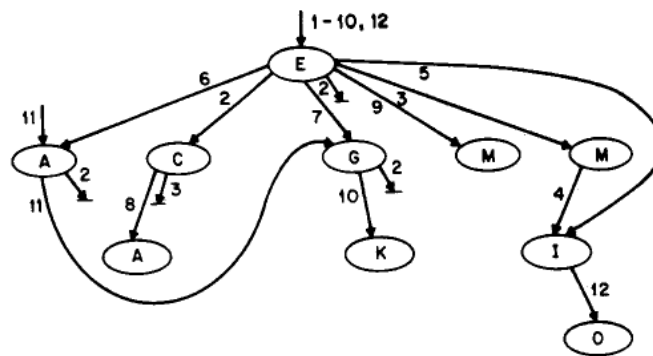


FIG. 4. A fully persistent search tree built using the fat node method, for the update sequence in Fig. 3. The version stamps and original null pointers of nodes are omitted as unnecessary.

pointer to a node $\bar{x}$ in position i if and only if node $\bar{x}$ contains name a with version stamp i . (This property follows from the explicit setting of access pointer inverses after every update operation in the augmented ephemeral structure.)

Figure 5 illustrates a fully persistent search tree with inverse pointers built using the fat node method. In this application, it is useful to make all pointers in a deleted ephemeral node null when the deletion takes place. This guarantees that every node in the nonaugmented ephemeral structure has at most one incoming pointer. (Otherwise it could have additional incoming pointers from deleted nodes.)

One more observation about the fat node structure is useful. Suppose that a fat node $\bar{x}$ contains a value v having field name f and valid interval I . Let i be any version in I other than the first (the version stamp of v). If we add to $\bar{x}$ a (redundant) copy of v having field name f and version stamp i , we affect neither the navigational correspondence between the ephemeral structure and the persistent structure nor the symmetry properties; the two copies of v have valid intervals that partition I .

3.3.3. The Split Node Data Structure

Having discussed inverse pointers, we are ready to focus on the node-splitting method itself. The effect of node splitting is to divide each fat node into one or more constant-size nodes, possibly introducing redundant field values of the kind described above. Let d be the number of pointer fields in an unaugmented ephemeral node, and let $k = d + p$ be the number of pointer fields in an augmented ephemeral node. Each persistent node will contain a version stamp for itself, the

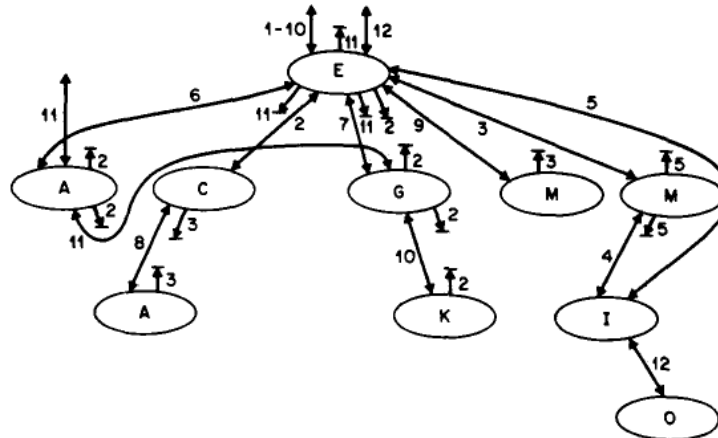


FIG. 5. A fully persistent search tree with inverse (parent) pointers built using the fat node method, for the update sequence in Fig. 3. Parent pointers leave the tops of nodes. Each double-headed arrow denotes two pointers, one the inverse of the other, with the same version stamp. The version stamps and original null pointers of nodes are omitted as unnecessary.

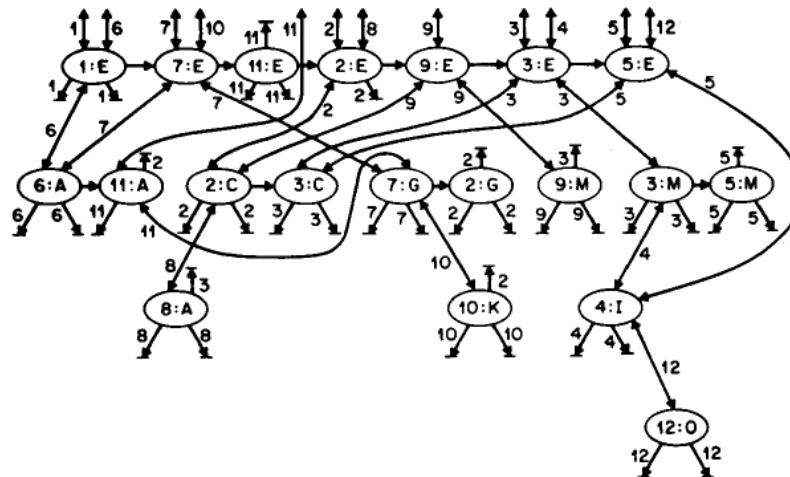


FIG. 6. A fully persistent search tree with inverse (parent) pointers built using the node-splitting method, for the update sequence in Fig. 3. Each pointer is numbered with its version stamp; the nodes are also numbered with their version stamps. The copy pointers leave the right sides of nodes and enter the left sides. As in Fig. 5, pointer fields of deleted nodes are made null during the deletion.

added to S . The time to make each inspected pointer proper is proportional to one plus the number of nodes created in Step 2.

It follows that the total time spent during an update operation is $O(1)$ per update step plus $O(1)$ per newly created node. We shall derive an $O(1)$ amortized bound on the number of nodes created per update step, thus obtaining an $O(1)$ amortized bound on the time and space per update step.

We define the potential of the split node structure to be $e/(e-k+1)$ times the number of persistent nodes minus $1/(e-k+1)$ times the number of vacant extra pointer fields, counting only at most e vacant extra pointer fields per node. (Recall that we require $e \geq k$.) We define the amortized space cost of an update step to be the number of new nodes the step creates plus the net increase in potential it causes. Because the initial potential is zero and the potential is always nonnegative, the total number of nodes in the split node structure is at most the total amortized space cost of the update operations.

We complete the analysis by showing that the amortized space cost of an update operation is linear in the number of update steps. Consider an update operation that performs u update steps. Phase one of the update operation has an amortized space cost of $O(u)$ and creates at most $2u$ new nodes. Let l be the number of new nodes created during phase two. Each new node created during phase one or two can ultimately result in the creation of up to k new pointers (one new incoming pointer corresponding to each of its k original outgoing pointers). Creation of a new node in phase two adds zero to the potential. The amortized space cost of phase two is thus at most l (for the newly created nodes) plus $(2u+l)k/(e-k+1)$

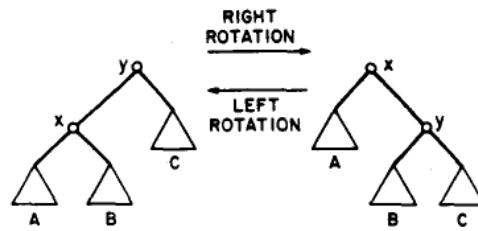


FIG. 7. A rotation in a binary tree. The tree shown can be a subtree of a larger tree.

and deletion algorithms. To perform an insertion, we follow the access path for the item to be inserted until we reach a missing node. At the location of the missing node we insert a new node containing the new item. We color the new node red. This may violate the red constraint, since the parent of the new node may be red. In this case we bubble the violation up the tree by repeatedly applying the recoloring transformation of Fig. 8a until it no longer applies. This either eliminates the violation or produces a situation in which one of the transformations in Figs. 8b, c, and d applies, each of which leaves no violation.

A deletion is similar. We first search for the item to be deleted. If it is in a node with a left child, we swap this node with the node preceding it in symmetric order, which we find by starting at the left child and following successive right pointers

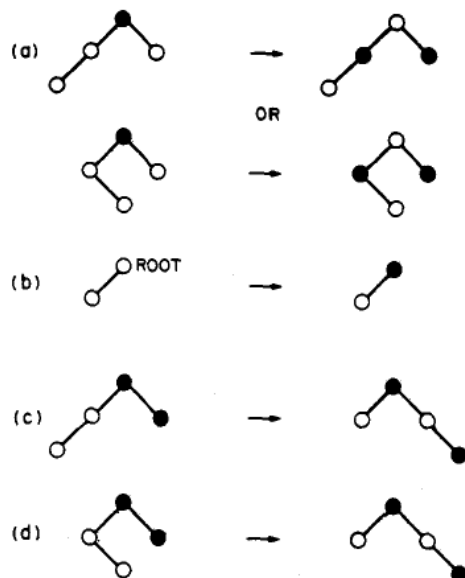


FIG. 8. The rebalancing transformations in red-black tree insertion. Symmetric cases are omitted. Solid nodes are black; hollow nodes are red. All unshown children of red nodes are black. In cases (c) and (d) the bottommost black node shown can be missing.

until reaching a node with no right child. Now the item to be deleted is in a node with at most one child. We replace this node by its only child (if any). This does not affect the red constraint but will violate the black constraint if the deleted node is black. If there is a violation, the replacing node (which may be missing) is *short*: paths from it to missing nodes contain one fewer black node than paths from its sibling. If the short node is red we merely color it black. Otherwise, we bubble the shortness up the tree by repeating the recoloring transformation of Fig. 9a until it no longer applies. Then we perform the transformation in Fig. 9b if it applies, followed if necessary by one application of 9c, d, or e.

An insertion requires $O(\log n)$ recolorings plus at most two rotations; a deletion, $O(\log n)$ recolorings plus at most three rotations. Furthermore the amortized number of recolorings per update operation is $O(1)$, i.e., m update operations produce $O(m)$ recolorings [15, 16, 20].

4.1. Partial Persistence

We can make a red-black tree partially persistent by using the node copying method of Section 2 with e (the number of extra pointers per node) equal to one. Because in the ephemeral structure there is only one access path to any node, we do not need inverse pointers. We obtain an additional simplification because colors are not used in access operations. Thus we do not need to save old node colors but can

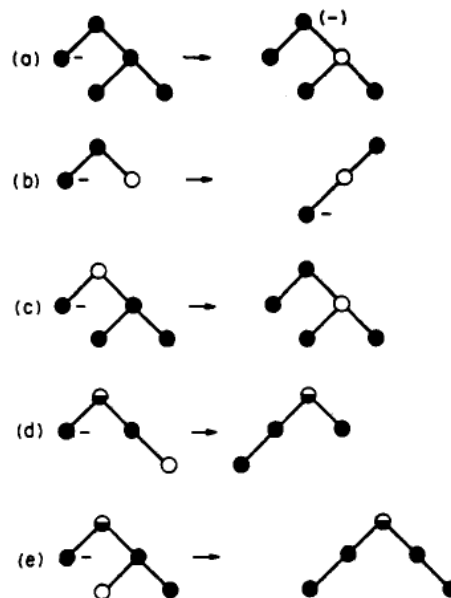


FIG. 9. The rebalancing transformations in red-black tree deletion. The two ambiguous (half-solid) nodes in (d) have the same color, as do the two in (e). Minus signs denote short nodes. In (a), the top node after the transformation is short unless it is the root.

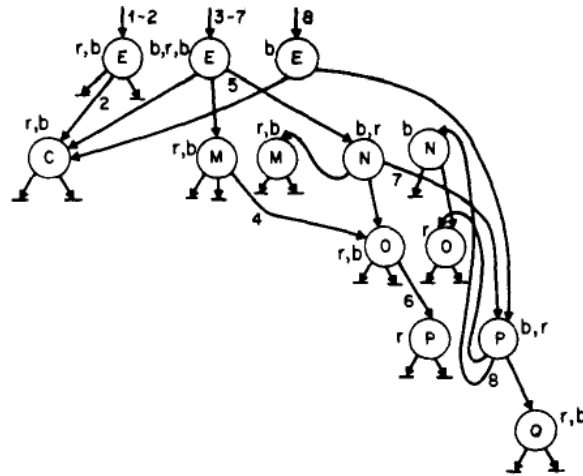


FIG. 10. A partially persistent red-black tree built using the node copying method of Section 2. The sequence of operations consists of insertions of E , C , M , O , N , P , Q , followed by deletion of M . Extra pointers are labeled with their version stamps. Each node is labeled with its successive colors. Insertion of item N triggers the transformation in Fig. 8d. Deletion of M triggers the transformation in Fig. 9d.

instead overwrite them. It is also easy to dispense with the copy pointers and the version stamps of the nodes. Thus each persistent node contains an item, three pointers, a color bit, and a version stamp and field name (left or right) for the third (extra) pointer. Figure 10 illustrates a partially persistent red-black tree.

A partially persistent red-black tree representing a history of m insertions and deletions occupies $O(m)$ space and allows $O(\log n)$ -time access to any version, where n is the number of items in the accessed version. The worst-case time per insertion or deletion is also $O(\log n)$. These trees and their applications, of which there are a number in computational geometry, are discussed more fully in a companion paper [29].

The node-copying method applies more generally to give an $O(m)$ -space partially persistent form of any balanced search tree with an $O(1)$ amortized update time for insertion and deletion. Such trees include weight-balanced trees [3, 24] and weak B -trees [15, 16, 20]. Increasing the number of extra pointers per node allows the addition of auxiliary pointers such as parent pointers and level links [4, 16]. Various kinds of *finger search trees* can be built using such extra pointers [4, 16, 17, 18, 35, 36]. (A finger search tree is a search tree in which access operations in the vicinity of certain preferred items, indicated by access pointers called *fingers*, are especially efficient.) By applying the node-copying method we can obtain partially persistent finger search trees occupying $O(m)$ space, with only a constant factor blowup in the amortized access and update times over their ephemeral counterparts.

4.2. Full Persistence

Let us turn to the problem of making a red-black tree fully persistent. We would like to obtain an $O(m)$ -space fully persistent red-black tree by using the node splitting method of Section 3. There are two difficulties, however. First, we must save old color bits; we are not free to overwrite them. Second, if we include recoloring time the $O(1)$ update time bound for ephemeral insertion and deletion is only amortized, whereas we need a worst-case $O(1)$ bound. Thus our goal becomes the construction of a variant of ephemeral red-black trees in which the worst-case update time per insertion or deletion, including recoloring, is $O(1)$.

To obtain such a variant, we use an idea of Tsakalidis [35, 36], *lazy recoloring*. Observe that the recoloring during a insertion or deletion affects only nodes along a single path in the tree and children of nodes along the path. Furthermore, the recoloring is extremely uniform except at the ends of the path. We exploit this observation by postponing all the recoloring except that at the path ends. We encode the postponed recoloring by storing certain information in the tree, information sufficient to allow us to perform the delayed recoloring incrementally, as it becomes necessary.

Consider an insertion. If we ignore $O(1)$ update steps at the beginning of the insertion and $O(1)$ update steps at the end, the rest of the updating consists of recoloring a path and siblings of nodes along the path as illustrated in Fig. 11: each node on the path has its color flipped, and the red siblings of nodes on the path become black. We call the configuration consisting of the path and the siblings to be recolored an *insertion interval*. The top and bottom nodes of the path are the *top* and *bottom* of the insertion interval, respectively. The siblings of nodes on the path that are not to be recolored are the *fringe nodes* of the interval. (The sibling of the top node is neither in the interval nor a fringe node.) Rather than actually recoloring an insertion interval, we store in the top node of the interval two pieces of information: a flag indicating the beginning of an insertion interval and the item in the bottom node of the interval. (Thus the top node contains two items, including its own.)

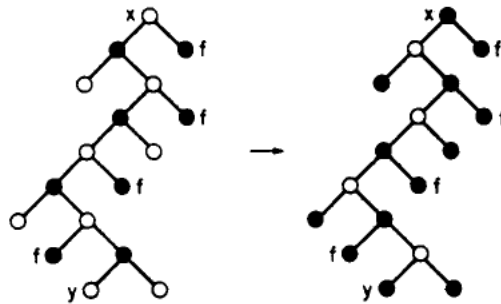


FIG. 11. An insertion interval with top node x and bottom node y , showing the desired recoloring. The nodes marked "f" are the fringe nodes of the interval. The interval is represented by storing in node x the item in node y (which is also still stored in y) along with a flag indicating that x is the top of an insertion interval.

In the case of a deletion all but $O(1)$ of the update time consists of recoloring siblings of nodes along a path as in Fig. 12: all the siblings, originally black, become red. We call the configuration consisting of the path and the siblings to be recolored a *deletion interval*. The top and bottom nodes of the path are the *top* and *bottom* of the interval, respectively. Rather than performing the recoloring in such an interval, we store in the top node of the interval two pieces of information: a flag indicating the beginning of a deletion interval and the item in the bottom node on the path.

We call insertion intervals and deletion intervals *recoloring intervals*. Our objective is to maintain the invariant that the recoloring intervals are vertex-disjoint. We shall show that this can be done using $O(1)$ node recolorings per insertion or deletion. Let us see what the disjointness of the recoloring intervals implies. Since during an access or update operation we always enter a recoloring interval via the top node, we always know whether we are in an interval and when we are entering or leaving one. (When entering an interval, we remember the extra item stored in the top node, which allows us to tell when we leave the interval.) Thus when we are in an interval we can keep track of the correct current node colors, which are not necessarily the same as their actual (unchanged) colors. This gives us the information we need to perform insertions and deletions.

To keep the recoloring intervals disjoint, we maintain insertion intervals so that their top and bottom nodes are actually red (but should be black), as in Fig. 11. This property can be imposed on an insertion interval not having it by coloring $O(1)$ nodes at the top and bottom of the interval, thereby shrinking it to an interval with the property. More generally, a recoloring interval can be shrunk by $O(1)$ nodes at either the top or the bottom by doing $O(1)$ recoloring. It can also be split in the middle into two recoloring intervals by doing $O(1)$ recoloring (see Fig. 13). Shrinking at the top of an interval requires moving the header information in the top node of the interval down to the new top node. Shrinking at the bottom requires changing the header information in the top node. Splitting an interval requires changing the header information in the top node of the original interval and adding header information to the new top node of the bottom half of the interval. In all cases the update time is $O(1)$.

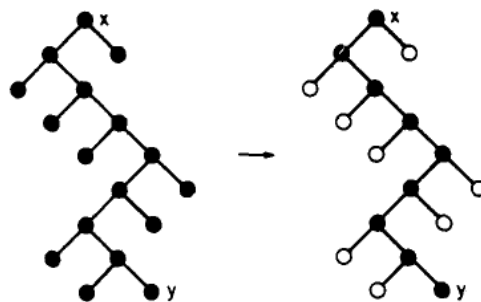


FIG. 12. A deletion interval with top node x and bottom node y , showing the desired recoloring. The interval is represented by storing in node x the item in node y along with a flag indicating that x is the top of a deletion interval.

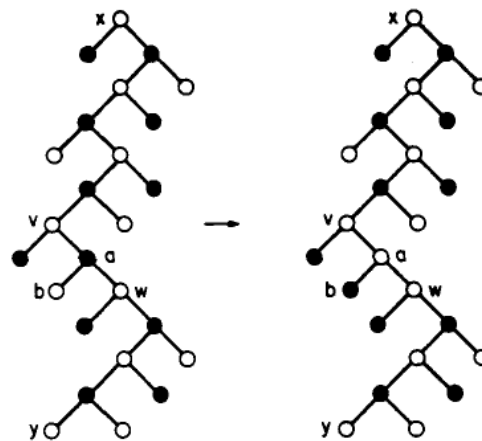


FIG. 13. Splitting an insertion interval. The nodes in the interval have their uncorrected colors. Recoloring the nodes *a* and *b* splits the insertion interval from *x* to *y* into two insertion intervals, one from *x* to *v* and one from *w* to *y*.

We maintain two invariants on recoloring intervals besides vertex disjointness: (i) every recoloring interval contains at least two nodes, and (ii) every fringe node of an insertion interval has a correct color of black. When a recoloring interval shrinks to one node, we correctly color it. When a fringe node of an insertion interval has its correct color changed from black to red, we split the insertion interval so that the node is no longer a fringe node. (A fringe node can be in another recoloring interval, but only as its top node.)

The crucial property of recoloring intervals is that they stop the propagation of color changes during an insertion or deletion. This is what allows us to keep the recoloring intervals disjoint while doing only $O(1)$ update steps per insertion or deletion.

In general, we represent a red-black tree as a collection of correctly colored nodes not in recoloring intervals and a vertex-disjoint set of recoloring intervals having the two properties listed above. To perform an insertion, we search from the tree root for the insertion location, keeping track of the bottommost recoloring interval on the way down the tree. We add the appropriate new red node to the tree. If the red constraint is violated, we walk up the tree implicitly applying the propagating insertion rule (Fig. 8a), without actually making the implied color changes, until either this rule is no longer applicable or applying the rule would recolor a node in a recoloring interval or a fringe node. There are only three ways in which the propagating insertion rule can recolor a node in a recoloring interval or a fringe node, each of which is terminal after at most one more transformation among those in Fig. 8 (see Fig. 14).

To complete the insertion, we shrink the recoloring intervals away from the vicinity of the terminating transformation sufficiently so that this transformation applies to nodes not in recoloring intervals and not fringe nodes. This may require

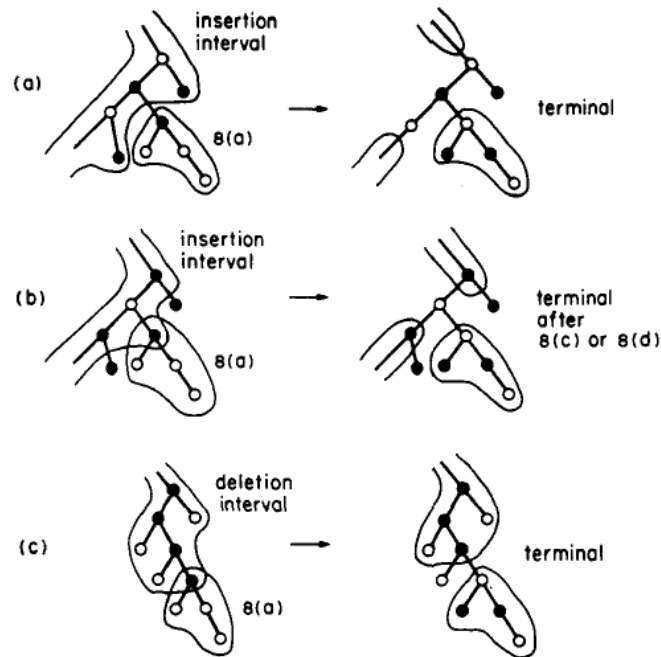


FIG. 14. The ways a propagating insertion can run into a recoloring interval. Recoloring intervals are shown correctly colored. In each case the transformation of Fig. 8a is to be applied. (a) Recoloring a fringe node. This terminates the insertion. The insertion interval splits. (b) Recoloring a node in an insertion interval. The next transformation is terminal. The interval splits. (c) Recoloring a node in a deletion interval. The recolored node must be the bottom of the interval. This terminates the insertion. The interval gets shorter.

splitting at most one recoloring interval, the bottommost along the original search path. We encode the remaining unperformed color changes as a single new insertion interval, disjoint from all other recoloring intervals. (Some of its fringe nodes may be the tops of other intervals.)

A deletion is similar. We search from the tree root for the item to be deleted, keeping track of the bottommost recoloring interval along the search path. If the item to be deleted is in a node with two children, we swap this node with its predecessor in symmetric order. (If either of the swapped nodes is the top or bottom of a recoloring interval, the header information in the top node of the interval may need to be updated.) Once the item to be deleted is in a node with only one child, we delete this node, first shrinking away from it any recoloring interval in the vicinity so that it is not affected by the deletion. We replace the deleted node by its child if it has one. Then we walk back up along the search path from the deleted node, implicitly applying the propagating deletion rule (Fig. 9a), until it no longer applies. This rule cannot recolor any node in a recoloring interval or any fringe node; furthermore, as soon as a node in a recoloring interval or a fringe node

tail. Let r be a displaced change record in a node $\bar{x}$ and let $\bar{z}$ be the tail of the displacement path of r . Let i be the version stamp of r , $\bar{y}$ the node preceding $\bar{z}$ in the displacement path of r , and s the record in $\bar{y}$ whose pointer to $\bar{z}$ is followed in traversing the displacement path of r . To shrink the displacement path of r , we create a new node $\bar{z}'$, regarded as a copy of $\bar{z}$, and add r to $\bar{z}'$ as its original record. We also create a copy s' of s , identical to s except that the pointer to $\bar{z}$ is replaced by a pointer to $\bar{z}'$, and insert s' in place of r as the change record in $\bar{x}$. Record s' gets version stamp i (see Fig. 15). After the shrinking, record r is no longer displaced; record s' is displaced unless $\bar{y} = \bar{x}$, and s' has a displacement path equal to the old path of r minus the tail. Node $\bar{z}'$ has no change record; thus the shrinking introduces a new place to put a change record.

The second, slightly more complicated operation on a displacement path is *splitting*. Let r be a displaced change record with version stamp i in a node $\bar{x}$, let $\bar{z}$ be a node in the body of the displacement path of r other than the tail, let $\bar{y}$ be the predecessor of $\bar{z}$ along the displacement path of r , let s be the record in $\bar{y}$ whose pointer to $\bar{z}$ is followed in traversing the displacement path, and let t be the record in $\bar{z}$ whose pointer is followed in continuing along the displacement path from $\bar{z}$. To split the path at $\bar{y}$, we create a new node $\bar{z}'$, regarded as a copy of $\bar{z}$, containing a copy of t as its original record and r with version stamp i as its change record. We also create a copy s' of s , replacing the pointer to $\bar{z}$ by a pointer to $\bar{z}'$, and insert s' in place of r as the change record of $\bar{x}$. Record s' gets version stamp i (see Fig. 16). This splits the old displacement path of r in two; the top half, from $\bar{x}$ to $\bar{y}$, is now the displacement path of s' ; the bottom half, from $\bar{z}'$ (instead of $\bar{z}$) to the old tail is the new displacement path of s . Both shrinking and splitting preserve invariants (i)–(iii).

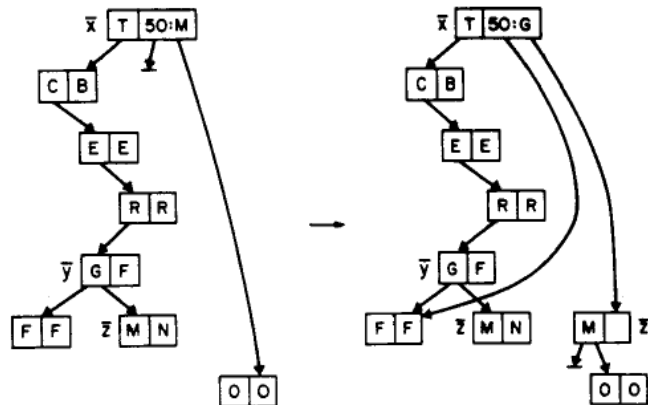


FIG. 15. Shrinking a displacement path. Irrelevant fields of nodes are omitted; all unshown version stamps of change records are assumed to be unrelated to 50 in the version tree. The displacement path with head $\bar{x}$ and tail $\bar{z}$ is shrunk by making a new copy $\bar{z}'$ of $\bar{z}$. The new displacement path is from $\bar{x}$ to $\bar{y}$.

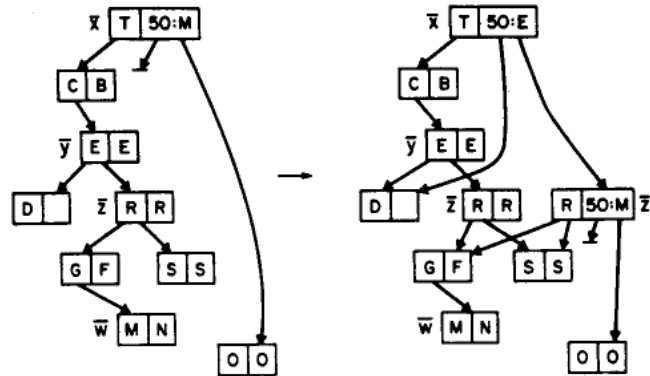


FIG. 16. Splitting a displacement path. Irrelevant fields of nodes are omitted; all unshown version stamps of change records are assumed to be unrelated to 50 in the version tree. The displacement path with head $\bar{x}$ and tail $\bar{w}$ is split at $\bar{y}$ by making a new copy $\bar{z}'$ of $\bar{z}$. The new displacement paths are from $\bar{x}$ to $\bar{y}$ and from $\bar{z}'$ to $\bar{w}$.

5.6. Update Operations

We are now ready to discuss the simulation of ephemeral update steps. For simplicity, we shall assume that each ephemeral update operation changes only one ephemeral node. The method is easily modified to handle any number of changes per update operation and, in particular, a constant number, as in red-black trees with lazy recoloring. Suppose we wish to simulate an ephemeral update that converts version i into the version j by changing node x . Let $\bar{x}$ be the persistent node corresponding to x . We construct a change record r with version stamp j containing the field values representing the new version of x . To find a place to store, r we back up along the access path to $\bar{x}$ in version i until we back up past the root of version i or we reach a node $\bar{y}$ such that either $\bar{y}$ has no change record or $\bar{y}$ is in the body of the displacement path of another change record s whose version stamp is an ancestor of j and such that s is in a node, say $\bar{z}$, on the access path to $\bar{x}$ in version i . There are four cases:

- (1) We back up past the root. In this case we create a new root node for version j . The new node contains as its original record a copy of the record in the old root whose pointers are followed in version i . As its change record, the new node contains r .
- (2) Node $\bar{y}$ contains no change record. We merely store r in $\bar{y}$ as its change record.
- (3) Node $\bar{y}$ is the tail of the displacement path of s . We shrink the displacement path of s and store r in the newly created copy $\bar{y}'$ of $\bar{y}$ as its change record.
- (4) Node $\bar{y}$ is in the body of the displacement path of s but is not the tail. We

⚠ Page is JS-rendered; the byte-graph diagrams are not in the static HTML. See the categorical-patches figures (next) for the same pushout/conflict model.

A Categorical Theory of Patches · 2013

<https://arxiv.org/pdf/1311.3903>

12 PAGE RENDERS

A Categorical Theory of Patches

Samuel Mimram Cinzia Di Giusto

CEA, LIST*

Abstract

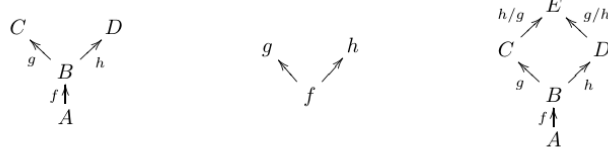
When working with distant collaborators on the same documents, one often uses a version control system, which is a program tracking the history of files and helping importing modifications brought by others as patches. The implementation of such a system requires to handle lots of situations depending on the operations performed by users on files, and it is thus difficult to ensure that all the corner cases have been correctly addressed. Here, instead of verifying the implementation of such a system, we adopt a complementary approach: we introduce a theoretical model, which is defined abstractly by the universal property that it should satisfy, and work out a concrete description of it. We begin by defining a category of files and patches, where the operation of merging the effect of two cointial patches is defined by pushout. Since two patches can be incompatible, such a pushout does not necessarily exist in the category, which raises the question of which is the correct category to represent and manipulate files in conflicting state. We provide an answer by investigating the free completion of the category of files under finite colimits, and give an explicit description of this category: its objects are finite sets labeled by lines equipped with a transitive relation and morphisms are partial functions respecting labeling and relations.

1 Introduction

It is common nowadays, when working with distant collaborators on the same files (multiple authors writing an article together for instance), to use a program which will track the history of files and handle the operation of importing modifications of other participants. These software called *version control systems* (VCS for short), like `git` or `Darcs`, implement two main operations. When a user is happy with the changes it has brought to the files it can record those changes in a *patch* (a file coding the differences between the current version and the last recorded version) and *commit* them to a server, called a *repository*. The user can also *update* its current version of the file by importing new patches added by other users to the repository and applying the corresponding modifications to the files. One of the main difficulties to address here is that there is no global notion of “time”: patches are only partially ordered. For instance consider a repository with one file A and two users u_1 and u_2 . Suppose that u_1 modifies file A into B by committing a patch f , which is then imported by u_2 , and

*This work was partially supported by the French project ANR-11-INSE-0007 REVER.

then u_1 and u_2 concurrently modify the file B into C (resp. D) by committing a patch g (resp. h). The evolution of the file is depicted on the left and the partial ordering of patches in the middle:



Now, suppose that u_2 imports the patch g or that u_1 imports the patch h . Clearly, this file resulting from the *merging* of the two patches should be the same in both cases, call it E . One way to compute this file, is to say that there should be a patch h/g , the *residual* of h after g , which transforms C into E and has the “same effect” as h once g has been applied, and similarly there should be a patch g/h transforming D into E . Thus, after each user has imported changes from the other, the evolution of the file is as pictured on the right above. In this article, we introduce a category $\mathcal{L}$ whose objects are files and morphisms are patches. Since residuals should be computed in the most general way, we formally define them as the arrows of pushout cocones, i.e. the square in the figure on the right should be a pushout.

However, as expected, not every pair of coinital morphisms have a pushout in the category $\mathcal{L}$: this reflects the fact that two patches can be conflicting (for instance if two users modify the same line of a file). Representing and handling such conflicts in a coherent way is one of the most difficult part of implementing a VCS (as witnessed for instance by the various proposals for Darcs: mergers, conflictors, graphictors, etc. [10]). In order to be able to have a representation for all conflicting files, we investigate the free completion of the category $\mathcal{L}$ under all pushouts, this category being denoted $\mathcal{P}$, which corresponds to adding all conflicting files to the category, in the most general way as possible. This category can easily be shown to exist for general abstract reasons, and one of the main contributions of this work is to provide an explicit description by applying the theory of presheaves. This approach paves the way towards the implementation of a VCS whose correctness is deduced from universal categorical properties.

Related work. The Darcs community has investigated a formalization of patches based on commutation properties [10]. *Operational transformations* tackle essentially the same issues by axiomatizing the notion of residual patches [9]. In both cases, the fact that residual should form a pushout cocone is never explicitly stated, excepting in informal sentences saying that “ g/f should have the same effect as g once f has been applied”. We should also mention another interesting approach to the problem using inverse semigroups in [4]. Finally, Houston has proposed a category with pushouts, similar to ours, in order to model conflicting files [3], see Section 6.

Plan of the paper. We begin by defining a category $\mathcal{L}$ of files and patches in Section 2. Then, in Section 3, we abstractly define the category $\mathcal{P}$ of conflicting files obtained by free finite cocompletion. Section 4 provides a concrete description of the construction in the simpler case where patches can only insert lines.

$d_i^n : n+1 \rightarrow n$ for every $n \in \mathbb{N}$ and $i \in [n+1]$ (respectively corresponding to insertion and deletion of a line at i -th position), subject to the relations

$$s_i^{n+1} s_j^n = s_{j+1}^{n+1} s_i^n \quad d_i^n s_i^n = \text{id}_n \quad d_i^n d_j^{n+1} = d_j^n d_{i+1}^{n+1} \quad (1)$$

whenever $0 \leq i \leq j < n$.

We will also consider the subcategory $\mathcal{L}^+$ of $\mathcal{L}$, with same objects, and *total* injective increasing functions as morphisms. This category models patches where the only possible operation is the insertion of lines: Proposition 2 can be adapted to show that $\mathcal{L}^+$ is the free monoidal category containing morphisms $\eta_a : I \rightarrow a$ and, in the unlabeled case, Proposition 4 can be similarly adapted to show that it is the free category generated by morphisms $s_i^n : n \rightarrow n+1$ satisfying $s_i^{n+1}s_j^n = s_{j+1}^{n+1}s_i^n$ with $0 \leq i \leq j < n$.

3 Towards a category of conflicting files

Suppose that A is a file which is edited by two users, respectively applying patches $f_1 : A \rightarrow A_1$ and $f_2 : A \rightarrow A_2$ to the file. For instance,

$$a \quad c \quad c \quad b \quad \xleftarrow{f_1} \quad a \quad b \quad \xrightarrow{f_2} \quad a \quad b \quad c \quad d \quad (2)$$

Now, each of the two users imports the modification from the other one. The resulting file, after the import, should be the smallest file containing both modifications on the original file: *acbcd*. It is thus natural to state that it should be a pushout of the diagram (2). Now, it can be noticed that not every diagram in $\mathcal{L}$ has a pushout. For instance, the diagram

$$a \quad c \quad b \quad \xleftarrow{f_1} \quad a \quad b \quad \xrightarrow{f_2} \quad a \quad d \quad b \quad (3)$$

does not admit a pushout in $\mathcal{L}$. In this case, the two patches f_1 and f_2 are said to be *conflicting*.

In order to represent the state of files after applying two conflicting patches, we investigate the definition of a category $\mathcal{P}$ which is obtained by completing the category $\mathcal{L}$ under all pushouts. Since, this completion should also contain an initial object (i.e. the empty file), we are actually defining the category $\mathcal{P}$ as the free completion of $\mathcal{L}$ under finite colimits: recall that a category is finitely cocomplete (has all finite colimits) if and only if it has an initial object and is closed under pushouts [6]. Intuitively, this category is obtained by adding files whose lines are not linearly ordered, but only partially ordered, such as on the left of

a	<<<<<< HEAD
c	=====
d	>>>>>> 5c55...
b	b

(4)

which would intuitively model the pushout of the diagram (3) if it existed, indicating that the user has to choose between c and d for the second line. Notice the similarities with the corresponding textual notation in `git` on the right. The name of the category $\mathcal{L}$ reflects the facts that its objects are files whose lines are linearly ordered, whereas the objects of $\mathcal{P}$ can be thought as files whose lines are only partially ordered. More formally, the category is defined as follows.

Definition 5. The category $\mathcal{P}$ is the *free finite conservative cocompletion* of $\mathcal{L}$: it is (up to equivalence of categories) the unique finitely cocomplete category together with an embedding functor $y : \mathcal{L} \rightarrow \mathcal{P}$ preserving finite colimits, such that for every finitely cocomplete category $\mathcal{C}$ and functor $F : \mathcal{L} \rightarrow \mathcal{C}$ preserving finite colimits, there exists, up to unique isomorphism, a unique functor $\tilde{F} : \mathcal{P} \rightarrow \mathcal{C}$ preserving finite colimits and satisfying $\tilde{F} \circ y = F$:

$$\begin{array}{ccc} \mathcal{L} & \xrightarrow{F} & \mathcal{C} \\ y \downarrow & \nearrow \tilde{F} & \\ \mathcal{P} & & \end{array}$$

Above, the term *conservative* refers to the fact that we preserve colimits which already exist in $\mathcal{L}$ (we will only consider such completions here). The “standard” way to characterize the category $\mathcal{P}$, which always exists, is to use the following folklore theorem, often attributed to Kelly [5, 1]:

Theorem 6. *The conservative cocompletion of the category $\mathcal{L}$ is equivalent to the full subcategory of $\hat{\mathcal{L}}$ whose objects are presheaves which preserve finite limits, i.e. the image of a limit in $\mathcal{L}^{\text{op}}$ (or equivalently a colimit in $\mathcal{L}$) is a limit in **Set** (and limiting cones are transported to limiting cones). The finite conservative cocompletion $\mathcal{P}$ can be obtained by further restricting to presheaves which are finite colimits of representables.*

Example 7. The category **FinSet** of finite sets and functions is the conservative cocompletion of the terminal category **1**.

We recall that the category $\hat{\mathcal{L}}$ of *presheaves* over $\mathcal{L}$, is the category of functors $\mathcal{L}^{\text{op}} \rightarrow \mathbf{Set}$ and natural transformations between them. The *Yoneda functor* $y : \mathcal{L} \rightarrow \hat{\mathcal{L}}$ defined on objects $n \in \mathcal{L}$ by $yn = \mathcal{L}(-, n)$, and on morphisms by postcomposition, provides a full and faithful embedding of $\mathcal{L}$ into the corresponding presheaf category, and can be shown to corestrict into a functor $y : \mathcal{L} \rightarrow \mathcal{P}$ [1]. A presheaf of the form yn for some $n \in \mathcal{L}$ is called *representable*.

Extracting a concrete description of the category $\mathcal{P}$ from the above proposition is a challenging task, because we a priori need to characterize firstly all diagrams admitting a colimit in $\mathcal{L}$, and secondly all presheaves in $\hat{\mathcal{L}}$ which preserve those diagrams. This paper introduces a general methodology to build such a category. In particular, perhaps a bit surprisingly, it turns out that we have to “allow cycles” in the objects of the category $\mathcal{P}$, which will be described as *the category whose objects are finite sets labeled by lines together with a transitive relation and morphisms are partial functions respecting labels and relations*.

4 A cocompletion of files and insertions of lines

In order to make our presentation clearer, we shall begin our investigation of the category $\mathcal{P}$ in a simpler case, which will be generalized in Section 6: we compute the free finite cocompletion of the category $\mathcal{L}^+$ (patches can only insert lines) in the case where the set of labels is a singleton. *To further lighten notations, in this section, we simply write $\mathcal{L}$ for this category.*

We sometimes characterize the objects in $\mathcal{L}$ as finite colimits of objects in a subcategory $\mathcal{G}$ of $\mathcal{L}$. This category $\mathcal{G}$ is the full subcategory of $\mathcal{L}$ whose objects

Proposition 8. Given a functor $F : \mathcal{C} \rightarrow \mathcal{D}$, with $\mathcal{D}$ cocomplete, the associated nerve $N_F : \mathcal{D} \rightarrow \hat{\mathcal{C}}$ admits a left adjoint $R_F : \hat{\mathcal{C}} \rightarrow \mathcal{D}$ called the realization along F . This functor is defined on objects $P \in \hat{\mathcal{C}}$ by

$$R_F(P) = \operatorname{colim}(\operatorname{El}(P) \xrightarrow{\pi} \mathcal{C} \xrightarrow{F} \mathcal{D})$$

Proof. Given a presheaf $P \in \hat{\mathcal{C}}$ and an object D , it can be checked directly that morphisms $P \rightarrow N_FD$ in $\hat{\mathcal{C}}$ with cocones from $\operatorname{El}(P) \xrightarrow{D}$ to D , which in turn are in bijection with morphisms $R_F(P) \rightarrow D$ in $\mathcal{D}$, see [7]. $\square$

Definition 9. A functor $F : \mathcal{C} \rightarrow \mathcal{D}$ is *dense* if it satisfies one of the two equivalent conditions:

- (i) the associated nerve functor $N_F : \mathcal{D} \rightarrow \hat{\mathcal{C}}$ is full and faithful,
- (ii) every object of $\mathcal{D}$ is canonically a colimit of objects in $\mathcal{C}$: for every $D \in \mathcal{D}$,

$$D \cong \operatorname{colim}(\operatorname{El}(N_FD) \xrightarrow{\pi} \mathcal{C} \xrightarrow{F} \mathcal{D}) \quad (5)$$

Since the functor I is dense, every object of $\mathcal{L}$ is a finite colimit of objects in $\mathcal{G}$, and $\mathcal{G}$ does not have any non-trivial colimit. One could expect the free conservative finite cocompletion of $\mathcal{L}$ to be the free finite cocompletion $\mathcal{P}$ of $\mathcal{G}$. We will see that this is not the case because the image in $\mathcal{L}$ of a non-trivial diagram in $\mathcal{G}$ might still have a colimit. By Theorem 6, the category $\mathcal{P}$ is the full subcategory of $\hat{\mathcal{L}}$ of presheaves preserving limits, which we now describe explicitly. This category will turn out to be equivalent to a full subcategory of $\hat{\mathcal{G}}$ (Theorem 15). We should first remark that those presheaves satisfy the following properties:

Proposition 10. Given a presheaf $P \in \hat{\mathcal{L}}$ which is an object of $\mathcal{P}$,

1. the underlying graph of P is finite,
2. for each non-empty path $x \twoheadrightarrow y$ there exists exactly one edge $x \rightarrow y$ (in particular there is at most one edge between two vertices),
3. $P(n+1)$ is the set of paths of length n in the underlying graph of P , and $P(0)$ is reduced to one element.

Proof. We suppose given a presheaf $P \in \mathcal{P}$, it preserves limits by Theorem 6. The diagram on the left

$$\begin{array}{ccc} & 3 & \\ s_2^2 \nearrow & & \nwarrow s_0^2 \\ 2 & & 2 \\ s_0^1 \nwarrow & 1 & \nearrow s_1^1 \end{array} \quad \begin{array}{ccccc} & P(s_2^2) & P(3) & P(s_0^2) & \\ & \nearrow & & \nwarrow & \\ P(2) & & & & P(2) \\ & \nwarrow & & \nearrow & \\ & P(s_0^1) & P(1) & P(s_1^1) & \end{array}$$

is a pushout in $\mathcal{L}$, or equivalently the dual diagram is a pullback in $\mathcal{L}^{\operatorname{op}}$. Therefore, writing D for the diagram $2 \xleftarrow{s_0^1} 1 \xrightarrow{s_1^1} 2$ in $\mathcal{L}$, a presheaf $P \in \mathcal{P}$ should satisfy $P((\operatorname{colim} D)^{\operatorname{op}}) \cong \lim P(D^{\operatorname{op}})$, i.e. the above pushout diagram in $\mathcal{L}$ should be transported by P into the pullback diagram in **Set** depicted on the right of the above figure. This condition can be summarized by saying that P should satisfy

Consider a finite graph $G \in \hat{\mathcal{G}}$, by Lemma 12, it induces a diagram (8) admitting a colimit if there is a universal arrow $f : G \rightarrow N_I n$ with $n \in \mathcal{L}$. From this it follows that the graph is acyclic: otherwise, we would have a non-empty path $x \rightarrow x$ for some vertex x , which would imply $f(x) < f(x)$. Similarly, suppose that G is a graph with vertices x and y such that there is no path $x \rightarrow y$ or $y \rightarrow x$, and there is an universal morphism $f : G \rightarrow N_I n$ for some $n \in \mathcal{L}$. Suppose that $f(x) \leq f(y)$ (the case where $f(y) \leq f(x)$ is similar). We can define a morphism $g : G \rightarrow N_I(n+1)$ by $g(z) = f(z) + 1$ if there is a path $x \rightarrow z$, $g(y) = f(x)$ and $g(z) = f(z)$ otherwise. This morphism is easily checked to be well-defined. Since we always have $f(x) \leq f(y)$ and $g(x) > g(y)$, there is no morphism $h : N_I n \rightarrow N_I(n+1)$ such that $h \circ f = g$.

Conversely, given a finite acyclic connected graph G , the relation $\leq$ defined on morphisms by $x \leq y$ whenever there exists a path $x \rightarrow y$ is a total order. Writing n for the number of vertices in G , the function $f : G \rightarrow N_I n$, which to a vertex associates the number of vertices strictly below it wrt $\leq$, is universal in the sense of Lemma 12. $\square$

Proposition 14. *The free conservative finite cocompletion $\mathcal{P}$ of $\mathcal{L}$ is equivalent to the full subcategory of $\hat{\mathcal{L}}$ whose objects are presheaves P satisfying the conditions of Proposition 10.*

Proof. By Lemma 11, the category $\mathcal{P}$ is equivalent to the full subcategory of $\hat{\mathcal{L}}$ whose objects are presheaves preserving limits of diagrams of the form (8) generated by some graph $G \in \hat{\mathcal{G}}$ which admits a colimit, i.e. by Lemma 13 the finite graphs which are acyclic and connected. We write G_n for the graph with $[n]$ as vertices and edges $i \rightarrow (i+1)$ for $0 \leq i < n-1$. It can be shown that any acyclic and connected finite graph can be obtained from the graph G_n , for some $n \in \mathbb{N}$, by iteratively adding an edge $x \rightarrow y$ for some vertices x and y such that there exists a non-empty path $x \rightarrow y$. Namely, suppose given an acyclic and connected finite graph G . The relation $\leq$ on its vertices, defined by $x \leq y$ whenever there exists a path $x \rightarrow y$, is a total order, and therefore the graph G contains G_n , where n is the number of edges of G . An edge in G which is not in G_n is necessarily of the form $x \rightarrow y$ with $x \leq y$, otherwise it would not be acyclic. Since by Proposition 10, see (7), the diagram generated by a graph of the form



is preserved by presheaves in $\mathcal{P}$ (which corresponds to adding an edge between vertices at the source and target of a non-empty path), it is enough to show that presheaves in $\mathcal{P}$ preserve diagrams generated by graphs G_n . This follows again by Proposition 10, see (6). $\square$

One can notice that a presheaf $P \in \mathcal{P}$ is characterized by its underlying graph since $P(0)$ is reduced to one element and $P(n)$ with $n > 2$ is the set of paths of length n in this underlying graph: $P \cong I_*(I^*P)$. We can therefore simplify the description of the cocompletion of $\mathcal{L}$ as follows:

Theorem 15. *The free conservative finite cocompletion $\mathcal{P}$ of $\mathcal{L}$ is equivalent to the full subcategory of the category $\hat{\mathcal{G}}$ of graphs, whose objects are finite graphs such that for every non-empty path $x \rightarrow y$ there exists exactly one edge $x \rightarrow y$. Equivalently, it can be described as the category whose objects are finite sets equipped with a transitive relation $<$, and functions respecting relations.*

In this category, pushouts can be explicitly described as follows:

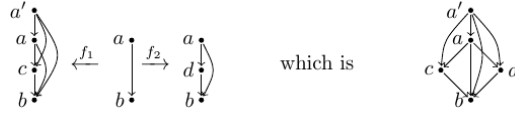
Proposition 16. *With the last above description, the pushout of a diagram in $\mathcal{P}(B, <_B) \xleftarrow{f} (A, <_A) \xrightarrow{g} (C, <_C)$ is $B \uplus C / \sim$ with $B \ni b \sim c \in C$ whenever there exists $a \in A$ with $f(a) = b$ and $g(a) = c$, equipped with the transitive closure of the relation inherited by $<_B$ and $<_C$.*

Lines with labels. The construction can be extended to the labeled case (i.e. L is not necessarily a singleton). The forgetful functor $\hat{\mathcal{L}} \rightarrow \mathbf{Set}$ sending a presheaf P to the set $P(1)$ admits a right adjoint $! : \mathbf{Set} \rightarrow \hat{\mathcal{L}}$. Given $n \in \mathbb{N}^*$ the elements of $!L(n)$ are words u of length n over L , with $!L(s_i^{n-1})(u)$ being the word obtained from u by removing the i -th letter. The free conservative finite cocompletion $\mathcal{P}$ of $\mathcal{L}$ is the slice category $\mathcal{L}/!L$, whose objects are pairs (P, ℓ) consisting of a finite presheaf $P \in \hat{\mathcal{L}}$ together with a *labeling* morphism $\ell : P \rightarrow !L$ of presheaves. Alternatively, the description of Proposition 15 can be straightforwardly adapted by labeling the elements of the objects by elements of L (labels should be preserved by morphisms), thus justifying the use of labels for the vertices in following examples.

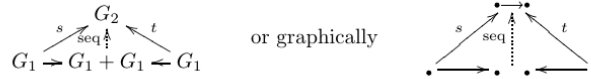
5 Examples

In this section, we give some examples of merging (i.e. pushout) of patches.

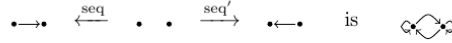
Example 17. Suppose that starting from a file ab , one user inserts a line a' at the beginning and c in the middle, while another one inserts a line d in the middle. After merging the two patches, the resulting file is the pushout of



Example 18. Write G_1 for the graph with one vertex and no edges, and G_2 for the graph with two vertices and one edge between them. We write $s, t : G_1 \rightarrow G_2$ for the two morphisms in $\mathcal{P}$. Since $\mathcal{P}$ is finitely cocomplete, there is a coproduct $G_1 + G_1$ which gives, by universal property, an arrow $\text{seq} : G_1 + G_1 \rightarrow G_2$:

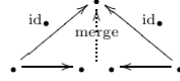


that we call the *sequentialization morphism*. This morphism corresponds to the following patch: given two possibilities for a line, a user can decide to turn them into two consecutive lines. We also write $\text{seq}' : G_1 + G_1 \rightarrow G_2$ for the morphism obtained similarly by exchanging s and t in the above cocone. Now, the pushout of



which illustrates how cyclic graphs appear in $\mathcal{P}$ during the cocompletion of $\mathcal{L}$.

Example 19. With the notations of the previous example, by taking the co-product of two copies of $\text{id}_{G_1} : G_1 \rightarrow G_1$, there is a universal morphism $G_1 + G_1 \rightarrow G_1$, which illustrates how two independent lines can be merged by a patch (in order to resolve conflicts).



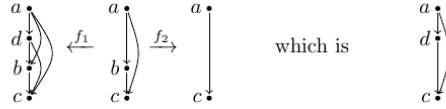
6 Handling deletions of lines

All the steps performed in previous sections in order to compute the free conservative finite cocompletion of the category $\mathcal{L}^+$ can be adapted in order to compute the cocompletion $\mathcal{P}$ of the category $\mathcal{L}$ as introduced in Definition 1, thus adding support for deletion of lines in patches. In particular, the generalization of the description given by Theorem 15 turns out to be as follows.

Theorem 20. *The free conservative finite cocompletion $\mathcal{P}$ of the category $\mathcal{L}$ is the category whose objects are triples $(A, <, \ell)$ where A is a finite set of lines, $<$ is a transitive relation on A and $\ell : A \rightarrow L$ associates a label to each line, and morphisms $f : (A, <_A, \ell_A) \rightarrow (B, <_B, \ell_B)$ are partial functions $f : A \rightarrow B$ such that for every $a, a' \in A$ both admitting an image under f , we have $\ell_B(f(a)) = \ell_A(a)$, and $a <_A a'$ implies $f(a) <_B f(a')$.*

Similarly, pushouts in this category can be computed as described in Proposition 16, generalized in the obvious way to partial functions.

Example 21. Suppose that starting from a file abc , one user inserts a line d after a and the other one deletes the line b . The merging of the two patches (in $\mathcal{P}'$) is the pushout of



i.e. the file adc . Notice that the morphism f_2 is partial: b has no image.

Interestingly, a category very similar to the one we have described in Theorem 20 was independently proposed by Houston [3] based on a construction performed in [2] for modeling asynchronous processes. This category is not equivalent to ours because morphisms are reversed partial functions: it is thus not the most general model (in the sense of being the free finite cocompletion). As a simplified explanation for this, consider the category **FinSet** which is the finite cocompletion of **1**. This category is finitely complete (in addition to cocomplete), thus **FinSet**^{op} is finitely cocomplete and **1** embeds fully and faithfully in it. However, **FinSet**^{op} is not the finite cocompletion of **1**. Another way to see this is that this category does not contain the “merging” morphism of Example 19, but it contains a dual morphism “duplicating” lines.

The same reasoning holds with s_1^1 instead of s_0^1 and we can therefore restrict ourselves to diagrams in which every object 2 is the target of exactly one arrow s_0^1 and one arrow s_1^1 .

Any such diagram K is obtained by gluing a finite number of diagrams of the form

$$\begin{array}{ccc} & 2 & \\ s_1^1 \nearrow & & \nwarrow s_0^1 \\ 1 & & 1 \end{array}$$

along objects 1, and is therefore of the form $\text{El}(G) \xrightarrow{\pi_G} \mathcal{G} \xrightarrow{I} \mathcal{L}$ for some finite graph $G \in \hat{\mathcal{G}}$: the objects of G are the objects 1 in K , the edges of G are the objects 2 in K and the source and target of an edge 2 are respectively given by the sources of the corresponding arrows s_1^1 and s_0^1 admitting it as target. For instance, the diagram on the left

$$\begin{array}{ccccccc} s_1^1 & 2 & s_0^1 & s_1^1 & 2 & s_0^1 & s_1^1 & 2 & s_0^1 \\ \nearrow & & \nwarrow & \nearrow & & \nwarrow & \nearrow & & \nwarrow \\ 1 & & 1 & & 1 & & 1 & & 1 \\ & & \searrow & & \swarrow & & \searrow & & \swarrow \\ & & s_1^1 & & s_0^1 & & s_1^1 & & s_0^1 \\ & & & 2 & & & & 2 & \end{array} \quad 0 \rightarrow 1 \rightarrow 2 \rightarrow 3$$

is generated by the graph on the right. $\square$

Lemma 12 *Given a graph $G \in \hat{\mathcal{G}}$, the associated diagram (8) admits a colimit in $\mathcal{L}$ if and only if there exists $n \in \mathcal{L}$ and a morphism $f : G \rightarrow N_I n$ in $\hat{\mathcal{L}}$ such that every morphism $g : G \rightarrow N_I m$ in $\hat{\mathcal{L}}$, with $m \in \mathcal{L}$, factorizes uniquely through $N_I n$:*

$$G \xrightleftharpoons[g]{f} N_I n \dashrightarrow N_I m$$

Proof. We have seen in proof of Proposition 8 that morphisms in $\hat{\mathcal{L}}(G, N_I n)$ are in bijection with cocones in $\mathcal{L}$ from $\text{El}(G) \xrightarrow{\pi_G} \mathcal{G} \xrightarrow{I} \mathcal{L}$ to n , and moreover given a morphism $h : n \rightarrow m$ in $\mathcal{G}$ the morphism $\hat{\mathcal{L}}(G, N_I n) \rightarrow \hat{\mathcal{L}}(G, N_I m)$ induced by post-composition with $N_I h$ is easily checked to correspond to the usual notion of morphism between n -cocones and m -cocones induced by $N_I h$ (every morphism $N_I n \rightarrow N_I m$ is of this form since N_I is full and faithful). We can finally conclude using the universal property defining colimiting cocones. $\square$

D Proofs for deletions of lines

In this section, we detail proofs of properties mentioned in Section 6.

D.1 Sets and partial functions

Before considering the conservative finite cocompletion of the category L , as introduced in Definition 1, it is enlightening to study the category **PSet** of sets and partial functions. A partial function $f : A \rightarrow B$ can always be seen

1. as a total function $f : A \rightarrow B \uplus \{\perp_A\}$ where $\perp_A$ is a fresh element wrt A , where given $a \in A$, $f(a) = \perp_A$ means that the partial function is undefined on A ,

D.2 A cocompletion of $\mathcal{L}$

The situation with regards to the category $\mathcal{L}$ is very similar. We follow the plan of Section 4 and first investigate the unlabeled case: $\mathcal{L}$ is the category with integers as objects and partial injective increasing functions $f : [m] \rightarrow [n]$ as morphisms $f : m \rightarrow n$.

We write $\mathcal{G}$ for the full subcategory of $\mathcal{L}$ whose objects are 0, 1 and 2. This is the free category on the graph

$$\begin{array}{ccccc} & & s_0^1 & & \\ & \xrightarrow{s_0^0} & 1 & \xrightarrow{s_1^1} & 2 \\ & \xleftarrow{d_0^0} & & \xleftarrow{d_1^1} & \\ & & d_0^1 & & \end{array}$$

subject to the relations

$$s_0^1 s_0^0 = s_1^1 s_0^0 = \text{id}_1 \quad d_0^0 s_0^0 = \text{id}_1 \quad d_0^1 s_0^1 = \text{id}_2 \quad d_1^1 s_1^1 = \text{id}_2 \quad d_0^0 d_0^1 = d_0^1 d_1^1 \quad (9)$$

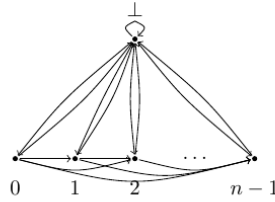
(see Proposition 4). We write $I : \mathcal{G} \rightarrow \mathcal{L}$ for the embedding and consider the associated nerve functor $N_I : \mathcal{L} \rightarrow \hat{\mathcal{G}}$. Suppose given an object $n \in \mathcal{L}$, the associated presheaf $N_I n$ can be described as follows. Its sets are

- $N_I n 0 = \mathcal{L}(I 0, n) \cong \{\star\}$,
- $N_I n 1 = \mathcal{L}(I 1, n) \cong [n] \uplus \{\perp\}$,
- $N_I n 2 = \mathcal{L}(I 2, n) \cong \{(i, j) \in [n] \times [n] \mid i < j\} \uplus \{(\perp, i) \mid i \in [n]\} \uplus \{(i, \perp) \mid i \in [n]\} \uplus \{(\perp, \perp)\}$:
a partial function $f : 2 \rightarrow n$ is characterized by the pair of images $(f(0), f(1))$ of 0, 1 $\in [n]$, where $\perp$ means undefined.

and morphisms are

- $N_I n s_0^0 : N_I n 1 \rightarrow N_I n 0$ is the constant function whose image is $\star$,
- $N_I n d_0^0 : N_I n 0 \rightarrow N_I n 1$ is the function whose image is $\perp$,
- $N_I n s_0^1 : N_I n 2 \rightarrow N_I n 1$ is the second projection,
- $N_I n s_1^1 : N_I n 2 \rightarrow N_I n 1$ is the first projection,
- $N_I n d_0^1 : N_I n 1 \rightarrow N_I n 2$ sends $i \in [n] \uplus \{\perp\}$ to $(\perp, i)$
- $N_I n d_1^1 : N_I n 1 \rightarrow N_I n 2$ sends $i \in [n] \uplus \{\perp\}$ to $(i, \perp)$

Such a presheaf can be pictured as a graph with $N_I n 1$ as set of vertices, $N_I n 2$ as set of edges, source and target being respectively given by the functions $N_I n s_1^1$ and $N_I n s_0^1$:



Its vertices are elements of $[n] \uplus \{\perp\}$ and edges are of the form

functors can be characterized as functors such that for every object $D \in \mathcal{D}$ the category is non-empty and connected (there is a zig-zag of morphisms between any two objects). A functor $F : \mathcal{C} \rightarrow \mathcal{D}$ is called a *discrete fibration* if for any object $C \in \mathcal{C}$ and morphism $g : D \rightarrow FC$ in $\mathcal{D}$ there exists a unique morphism $f : C' \rightarrow C$ in $\mathcal{C}$ such that $Ff = g$ called the lifting of g . To any such discrete fibration one can associate a presheaf $P \in \hat{\mathcal{D}}$ defined on any $D \in \mathcal{D}$ by $PD = F^{-1}(D) = \{C \in \mathcal{C} \mid FC = D\}$ and on morphisms $g : D' \rightarrow D$ as the function Pg which to $C \in PD$ associates the source of the lifting of g with codomain C . Conversely, any presheaf $P \in \hat{\mathcal{D}}$ induces a discrete fibration $\text{El}(P) \xrightarrow{\pi} \mathcal{D}$, and these two operations induce an equivalence of categories between the category $\hat{\mathcal{D}}$ and the category of discrete fibrations over $\mathcal{D}$. It was shown by Paré, Street and Walters [8, 11] that any functor $F : \mathcal{C} \rightarrow \mathcal{D}$ factorizes as final functor $J : \mathcal{C} \rightarrow \mathcal{E}$ followed by a discrete fibration $K : \mathcal{E} \rightarrow \mathcal{D}$, and this factorization is essentially unique: this is called the *comprehensive factorization* of a functor. More explicitly, the functor K can be defined as follows. The inclusion functor $\mathbf{Set} \rightarrow \mathbf{Cat}$ which send a set to the corresponding discrete category admits a left adjoint $\Pi_0 : \mathbf{Cat} \rightarrow \mathbf{Set}$, sending a category to its connected components (its set of objects quotiented by the relation identifying two objects linked by a zig-zag of morphisms). The discrete fibration part K above can be defined as $\text{El}(P) \xrightarrow{\pi} \mathcal{D}$ where $P \in \hat{\mathcal{D}}$ is the presheaf defined by $P = \Pi_0(-/F)$. In this precise sense, every diagram F in $\mathcal{D}$ is “equivalent” to one which is “generated” by a presheaf P on $\mathcal{D}$ (we adopted this informal terminology in the article in order to avoid having to introduce too many categorical notions).

In our case, we can thus restrict to diagrams in $\mathcal{L}$ generated by presheaves on $\mathcal{L}$. Finally, since $I : \mathcal{G} \rightarrow \mathcal{L}$ is dense, we can further restrict to diagrams generated by presheaves on $\mathcal{G}$ by interchange of colimits. $\square$

Lemma 13 applies almost as in Section 4: since the morphism $f : G \rightarrow N_{In}$ (seen as a partial functions between graphs) has to satisfy the universal property of Lemma 12, by choosing for every vertex x of G a partial function $g_x : G \rightarrow N_{Im}$ which is defined on x (such a partial function always exists), it can be shown that the function f has to be total. The rest of the proof can be kept unchanged. Similarly, Proposition 14 applies with proof unchanged.

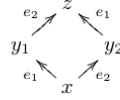
Finally, we have that

Theorem 32. *The free conservative finite cocompletion $\mathcal{P}$ of $\mathcal{L}$ is equivalent to the full subcategory of $\hat{\mathcal{L}}$ whose objects are presheaves P satisfying the conditions of Proposition 30. Since its objects P satisfy $I_*I^*(P) \cong P$, it can equivalently be characterized as the full subcategory of $\hat{\mathcal{G}}$ whose objects P are*

1. finite,
2. transitive: for each non-empty path $x \rightarrow y$ there exists exactly one edge $x \rightarrow y$,
3. pointed: $P(0)$ is reduced to one element.

From this characterization (which can easily be extended to the labeled case), along with the correspondence between pointed graphs and graphs with partial functions (Proposition 28), the category is shown to be equivalent to the

Definition 35. A *categorical event structure* (E, λ) in a category $\mathcal{C}$ with an initial object consists of an event structure equipped with a *labeling* functor $\lambda : (TE)^* \rightarrow \mathcal{C}$, where $(TE)^*$ is the free category generated by the graph TE , such that $\lambda\emptyset$ is the initial object of $\mathcal{C}$ and the image under λ of every square



in TE is a pushout in $\mathcal{C}$.

The following proposition shows that a categorical event structure is characterized by a suitable labeling of events of E by morphisms of $\mathcal{C}$.

Proposition 36. *The functor λ is characterized, up to isomorphism, by the image of the transitions $\downarrow e \xrightarrow{e} \downarrow e \uplus \{e\}$.*

We can now define a *repository* to be simply a finite categorical event structure $(E, \leq, \#, \lambda : T(E) \rightarrow \mathcal{L})$. Such a repository extends to a categorical event structure $(E, \leq, \#, I \circ \lambda : T(E) \rightarrow \mathcal{P})$, where $\#_0$ is the empty conflict relation. The *state* S of such an event structure is the file obtained as the image $S = I \circ \lambda(E)$ of the maximal configuration: this is the file that the users is currently editing given his repository. Usual operations on repositories can be modeled in this context, for instance importing the patches of another repository is obtained by a pushout construction (the category of repositories is finitely cocomplete).

Here, $++ : \text{History } n1 \ n2 \rightarrow \text{History } n2 \ n3 \rightarrow \text{History } n1 \ n3$ appends two histories. Then, if we have a pair of complete histories $h1, h2$, we reconcile them by returning a history h' which extends both $h1$ and $h2$. The suffixes of $h1$ and $h2$ yielding h' are the pair of converging patches produced by the merge.

```
mergeH : {n n1 n2 : Nat}
  (h1 : History 0 n1) (h2 : History 0 n2) →
  Σ (n' : Nat). Σ (h' : History 0 n').
    Extension h1 h' × Extension h2 h'
```

Once we have defined `mergeH`, we can convert its output back to paths. A complete history can be transformed into a path `doc []` = `doc h` by repeated concatenation:

```
toPath : {n : Nat} (h : History 0 n) → doc [] = doc h
toPath [] = refl
toPath (ADD s@1 :: h') = addP s 1 o toPath h'
toPath (RM 1 :: h') = rmP 1 o toPath h'
```

To turn an `Extension h h'` into a path, we need only travel from `doc h` to `doc []` and back to `doc h'`:

```
extToPath : {n n' : Nat}
  {h : History 0 n} {h' : History 0 n'} →
  Extension h h' → doc h = doc h'
extToPath _ = (toPath h') o !(toPath h)
```

`extToPath` completely ignores the extension itself; intuitively, this is possible because extensions are more informative than paths, since the former contain only compositions of generators.

Combining these ingredients, we define `merge` as:

```
merge p1 p2 =
  let (n', (h', (e1, e2))) =
    mergeH (interpH p1 []) (interpH p2 [])
  in (n', (h', (extToPath e1, extToPath e2)))
```

This shows that `merge` reduces to `mergeH`. We have not yet defined `mergeH`, but we can reduce it to defining a merge on simple text files. Because `History 0 n` is quotiented by patch laws, we must show that `mergeH` sends equal histories to equal results. One way to handle this is to choose a representative for each equivalence class of histories, and then compute on these representatives. Here, we can use the function `replay` to convert each `History 0 n` to its file contents in `Vec n String`. Then, we could define a merge function directly on files (perhaps using existing algorithms), and then compute extensions of the input histories which result in that those files. Such a `mergeH` would necessarily respect the patch laws because `replay` does.

7. Related Work

Several prior category-theoretic analyses of version control have been considered. Jacobson [16] interprets patches in inverse semigroups, where they are essentially partial bijections. Mimram and Di Giusto [28] analyze merging as a pushout, which provides a canonical merge for every pair of patches, including a primitive representation of merge conflicts. Houston [15] also discusses merge as pushout, and a duality with exceptions. Our contribution, relative to these analyses, is to present patch theory in a categorical setting that is also a programming formalism, so it directly leads to an implementation. These analyses consider settings where not all maps are invertible. In homotopy type theory all identity types are symmetric, and to fit patch theory into this symmetric setting, we either considered a language where all patches were naturally total bijections on any repository (Section 4 and 5), or used types to restrict patches to repositories where they are bijections (Section 6).

Dagit [10] presents an approach to proving some invariants of a version control implementation using advanced features of Haskell's type system. Camp (Commute And Merge Patches) [7] is

an experimental version control system based on Darcs; the Camp project aims to prove the correctness of its patch theory in Coq. We have not yet mechanized the programs described here, but our work provides another possible path to formalization.

Swierstra and Löh [34] explore the use of separation logic [30] for specifying the behavior of patches. In Section 6 we took repository contexts to be patch histories, but it would be interesting to consider using separation logic formulas to describe histories, which would allow for small-footprint specifications of patches.

8. Conclusion

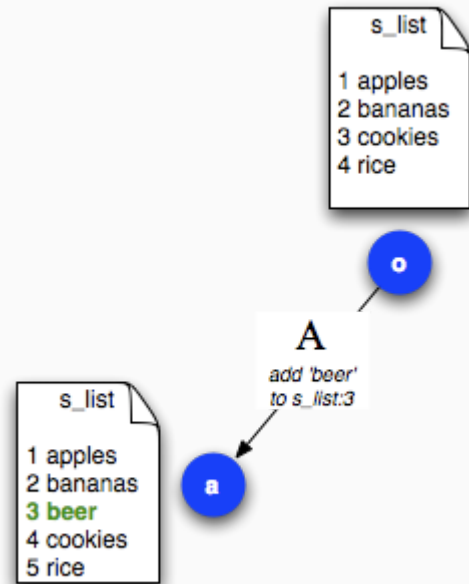
Inspired by the patch theory of Darcs [12], which emphasizes the groupoid structure of patches, we have explored the formulation of patch theory within the framework of homotopy type theory. Patch theories are given as higher inductive definitions in which we specify generators for the points (path contexts), 1-dimensional paths (patches), and 2-dimensional paths (patch laws). The groupoid laws come “for free”, and so need not be specified explicitly. An idealized implementation of a patch theory is given by a function mapping the patch theory into a univalent universe of sets and bijections. The sets are concrete repositories and the bijections are the actual actions of the patches on repositories. The mapping is intrinsically functorial, and hence must respect the intrinsic groupoid structure, but is given by the elimination principle for higher-inductive types, which demands that patches be realized by bijections satisfying the patch laws.

Besides these general structural considerations, some homotopy-theoretic concepts play a role in the development. In particular the “encode/decode” functions used to characterize identity types [23] [35] here become an implementation technique. For example, to define the merge of a span of patches (that is, a pair of paths with a common domain), we first pass to a concrete representation of paths, define the merge on the representation, and then pass back to a reconciliation, a cospan of patches (a pair of patches with common codomain) in the identity type. Other interpretations of a patch theory are definable in a similar manner, providing operations, such as logging, of practical interest for revision control.

There is much more to be done. Most importantly, the development of the merge operation in Section 6 is incomplete, because we have not proved the required properties of it. There we define `merge` using a partial characterization of the identity type of R as complete histories. Defining functions in either direction is sufficient to pass between these two representations, but not to prove properties of `merge` via properties of `mergeH`, such as the merge laws. For this, a more precise characterization is needed, namely an inductive type which is *equivalent* to the identity type, as $\mathbb{Z}$ is to the identity type of the circle. We leave this to future work.

More broadly, the computational interpretation of homotopy type theory must be developed further, including a fuller understanding of the interplay between definitional equality, propositional equality, and computation, and an understanding of what type erasure would mean in that setting. This includes developing a fuller understanding of “sub-homotopy” computation, meaning the mapping into contractible types.

One aspect of the homotopy framework that we have found limiting is the requirement that paths be symmetric (have inverses). In Darcs, inverses are used to define merging in terms of the elegant concept of pseudocommutation. But demanding inverses, rather than just retractions representing “undo” operations, is both conceptually questionable and practically problematic. We worked around the presence of inverses by refining contexts using patch histories, and using histories it was possible to define merge directly on only “forward” patches, rather than using inverses and pseudocommutation. Despite efforts, we were unable to formulate pseudocommutation in our setting, because to do so seems to re-



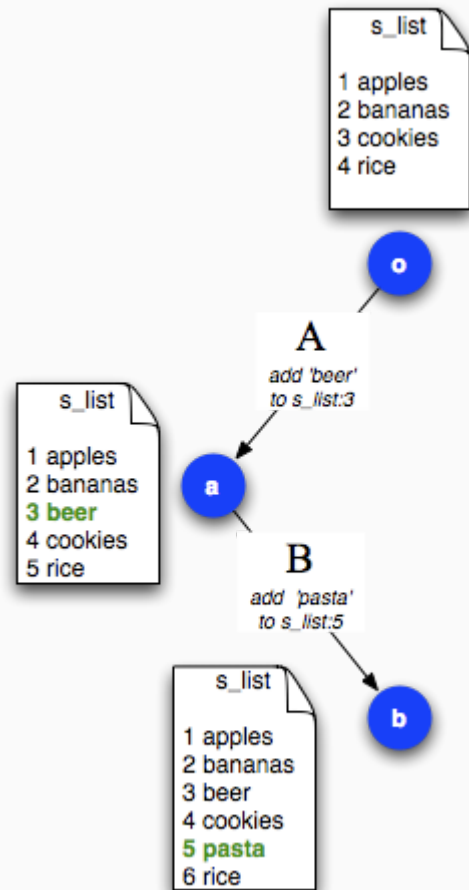
Understanding Darcs/Patch Theory · i03



Understanding Darcs/Patch Theory · i04



Understanding Darcs/Patch Theory · i05



Understanding Darcs/Patch Theory · i06

A

add beer to s_list:3

B

add pasta to s_list:5

commute

B₁

add pasta to s_list:4

A₁

add beer to s_list:3

B₁

add pasta to s_list:4

A₁

add beer to s_list:3

A₁⁻¹

remove beer from s_list:3

s_list

1 apples
2 bananas
3 beer
4 cookies
5 pasta
6 rice

s_list

1 apples
2 bananas
3 cookies
4 pasta
5 rice

B

move s_list to shopping

B⁻¹

move shopping to s_list

A

add 'beer' to s_list:3

commute

B

move s_list to shopping

A₁

add 'beer' to shopping:3

B₁⁻¹

move shopping to s_list

B

move s_list to shopping

A₁

add 'beer' to shopping:3

shopping

1 apples
2 bananas
3 beer
4 cookies
5 rice

s_list

1 apples
2 bananas
3 beer
4 cookies
5 rice

s_list

1 apples
2 bananas
3 beer
4 cookies
5 rice

Angle 3 — Versioned Databases

Mergeable Persistent Data Structures · 2015

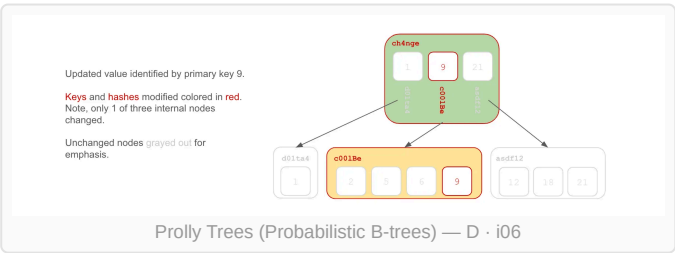
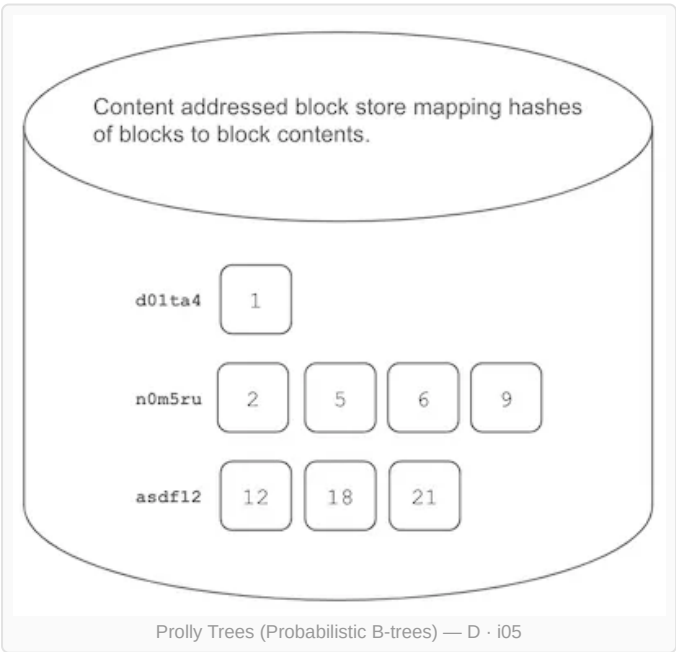
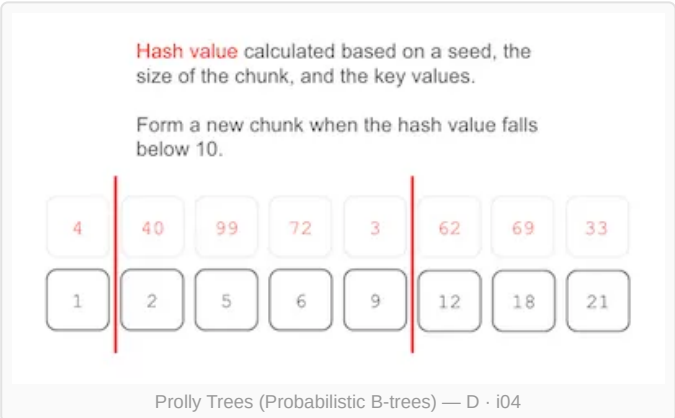
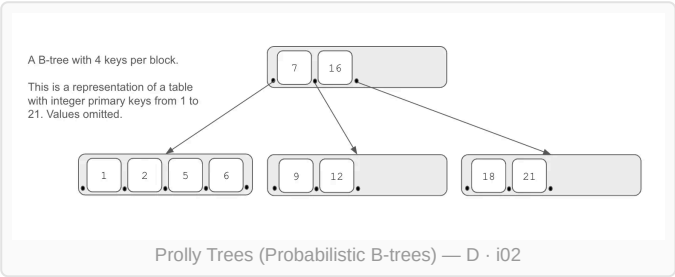
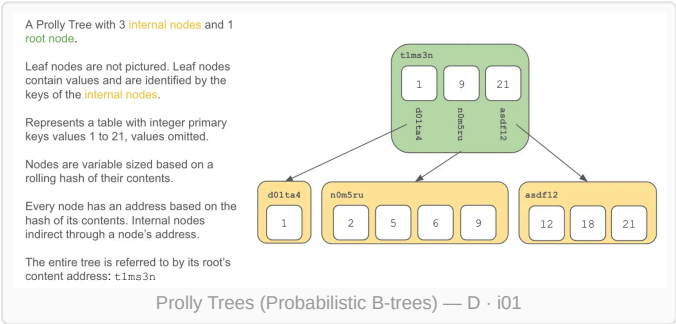
<https://blog.acolyer.org/2015/01/14/mergeable-persistent-data-structures/>

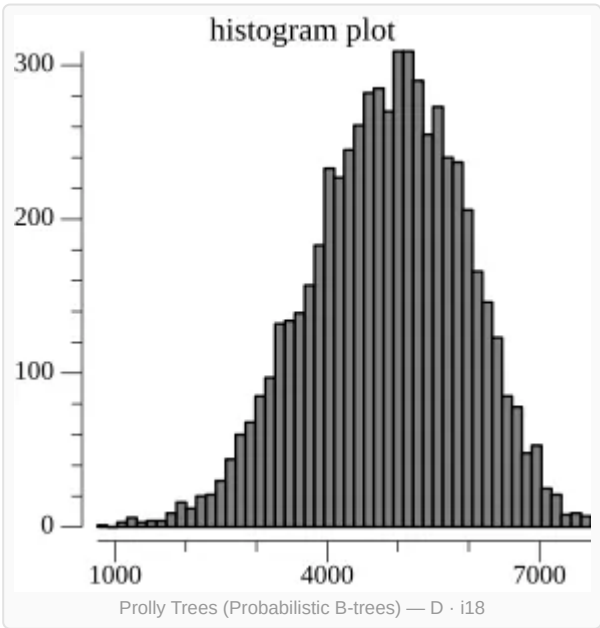
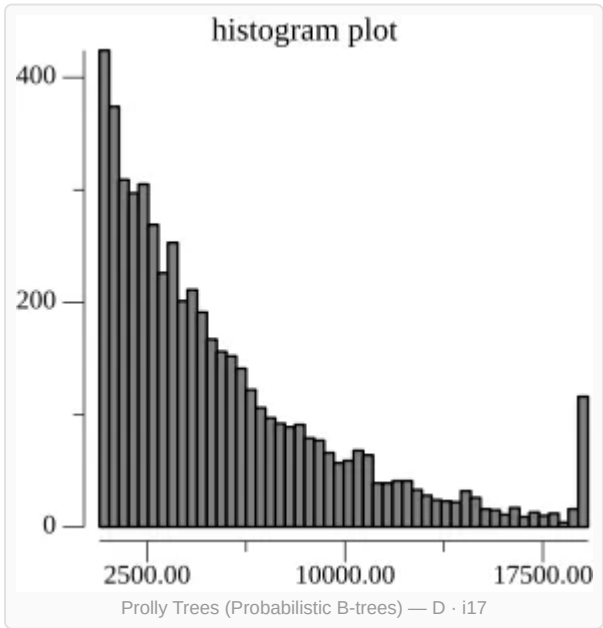
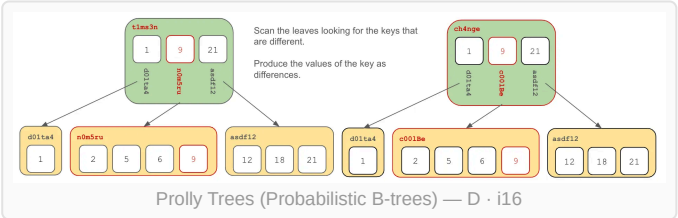
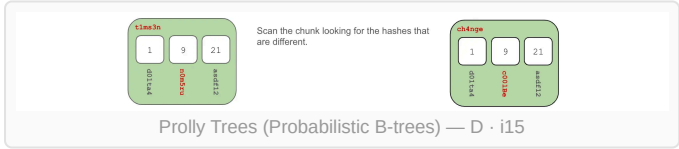
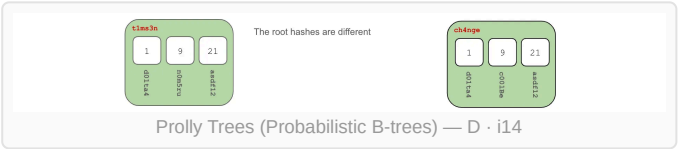
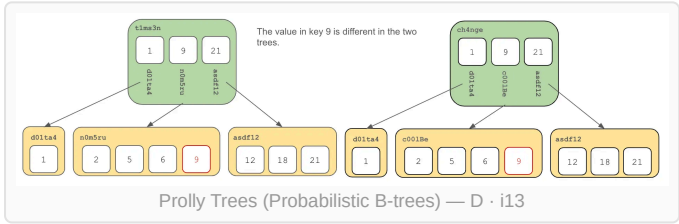
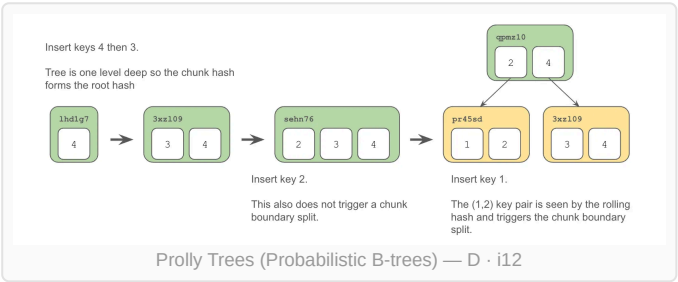
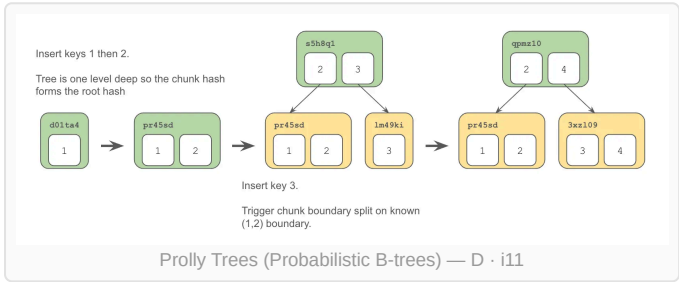
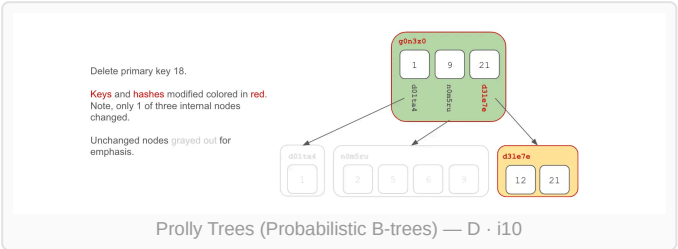
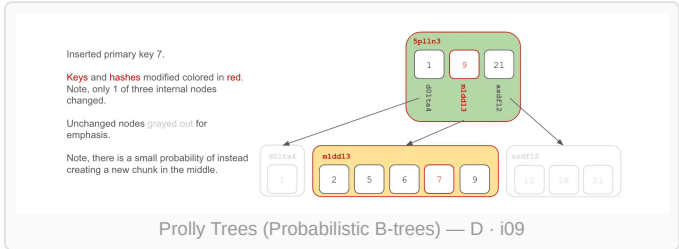
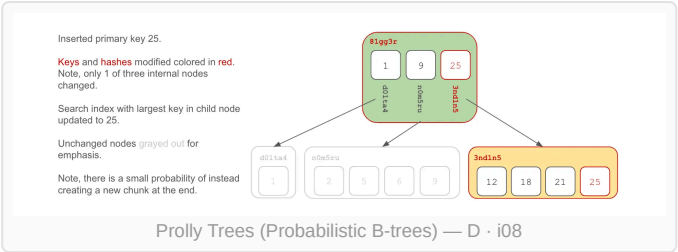
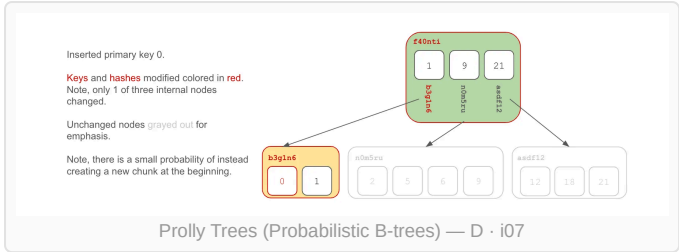
⚠ The Morning Paper summary page does not embed the source paper's figures as images.

Proilly Trees (Probabilistic B-trees) — Dolt Storage Engine Architecture · 2023

<https://docs.dolthub.com/architecture/storage-engine/prolly-tree>

18 EXTRACTED FIGURES





Version Controlled Databases: Defining a Category · 2020

<https://www.dolthub.com/blog/2020-11-17-version-control-databases-defining-a-category/>

2 EXTRACTED FIGURES

Google

version controlled database

X

🔍

All

News

Images

Videos

Shopping

More

Settings

Tools

About 319,000,000 results (0.55 seconds)

dbmstools.com › categories › version-control-tools

18 Database Version Control - DBMS Tools

DBV is a user-friendly tool for database version control. It provides schema management ensuring that you and your team always have the same schema structure ...

Repositories: File

Type: State-based tools

Runs on: (for desktop): Mac OS, Windows

www.red-gate.com › blog › database-devops › database...

Six reasons to version control your database - Redgate Software

May 10, 2019 — For most application developers, it's unthinkable to work without version control. What about the database? What's stopping you version ...

www.infoq.com › articles › Database-Version-Control

The Definitive Guide to Database Version Control - InfoQ

Dec 4, 2014 — Ensure all database code is covered (structure, code, reference content, grants); Ensure the version control repository acts as the single source of ...

www.liquibase.com › resources › guides › database-ver...

Database Version Control Guide | Database Versioning ...

Add version control to your database. Understand methods, requirements, and tools that will help you accelerate software delivery.

stackoverflow.com › questions › do-you-use-source-co...

Do you use source control for your database items? - Stack ...

Sep 22, 2008 — Must read Get your database under version control. Check the series of posts by K. Scott Allen. When it comes to version control, the database ...

58 answers

How to version control a record in a database ...

12 answers

Nov 27, 2008

What are the standard/recommended ways to store ...

3 answers

Dec 15, 2011

Database version control plan: hot or not? - Stack ...

1 answer

Dec 3, 2010

Testing and Managing database versions against code ...

7 answers

Aug 28, 2008

More results from stackoverflow.com

www.liquibase.org

Liquibase | Open Source Version Control for Your Database

Liquibase Community is an open source project that helps millions of developers rapidly track, version, and deploy database schema changes.

medium.com › database-source-control-versioning-and...

Database version control, versioning, and environment ...

Even infrastructure can be coded and versioned controlled. But some engineers are still oblivious to the need for version control around databases that house ...

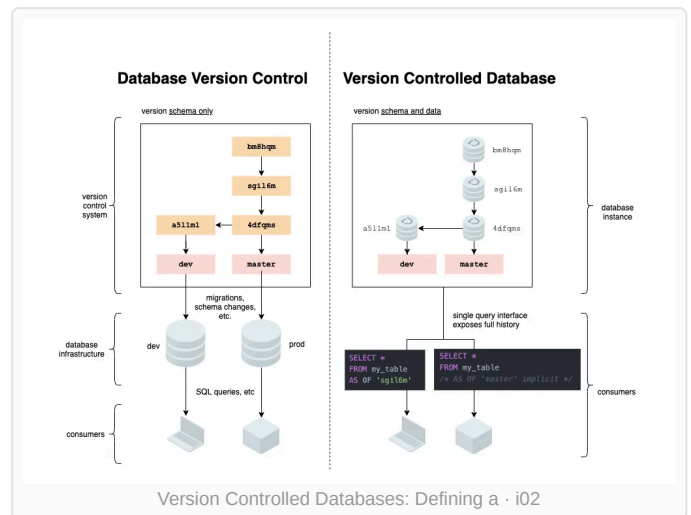
softwareengineering.stackexchange.com › questions › v...

Version controlling a database's contents - Software ...

Other databases may be more of a problem. Having the database enforce the rule will keep your code honest. You are quite correct that lots of small changes will ...

6 answers

Version Controlled Databases: Defining a · i01



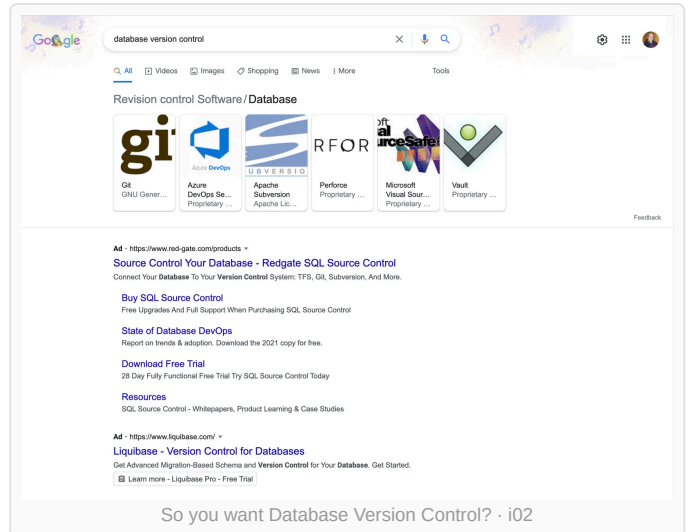
So you want Database Version Control? · 2021

<https://www.dolthub.com/blog/2021-09-17-database-version-control/>

3 EXTRACTED FIGURES



So you want Database Version Control? · i01



So you want Database Version Control? · i02



So you want Database Version Control? · i03

XTDB v1 Documentation — Bitemporal · ~2021 (v1 docs, undated page)

<https://v1-docs.xtdb.com/concepts/bitemporal/>

⚠ Bitemporal diagrams are rendered inline (HTML/SVG/JS), not as extractable `<img>` figures.

Angle 4 — Graph & RDF Versioning

TerminusDB Explanation (What is TerminusDB? / Graphs Explanation + supporting docs) · 2020-2025

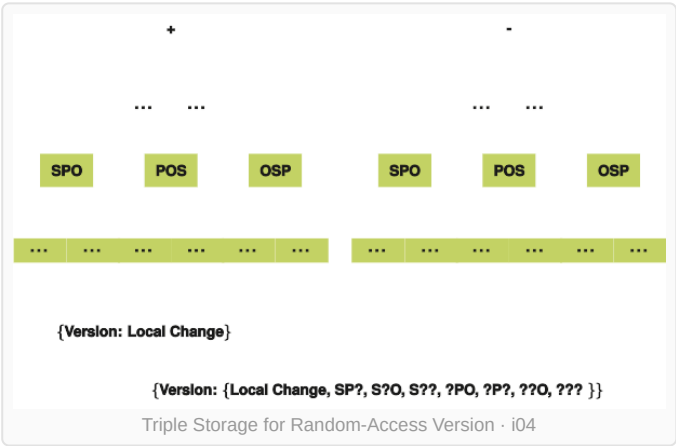
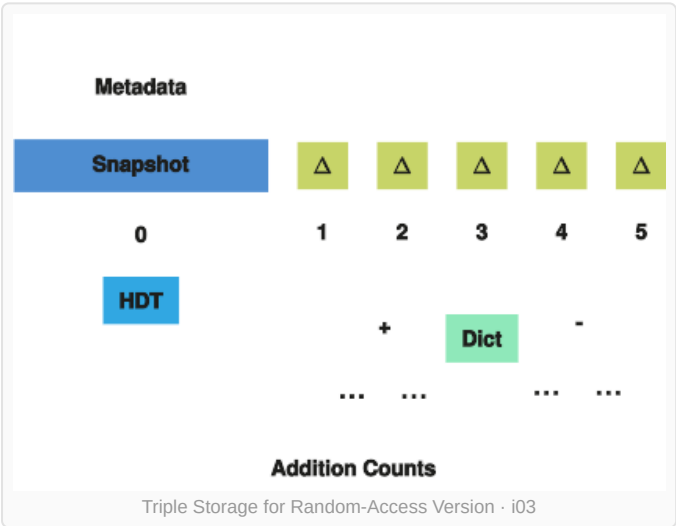
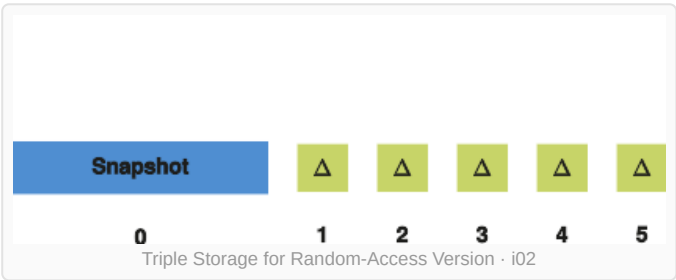
<https://terminusdb.org/docs/terminusdb-explanation/>

⚠ Doc figures are lazy-loaded/inline; not extractable as static images.

Triple Storage for Random-Access Versioned Querying of RDF Archives (OSTRICH) · 2018

<https://rdfostrich.github.io/article-jws2018-ostrich/>

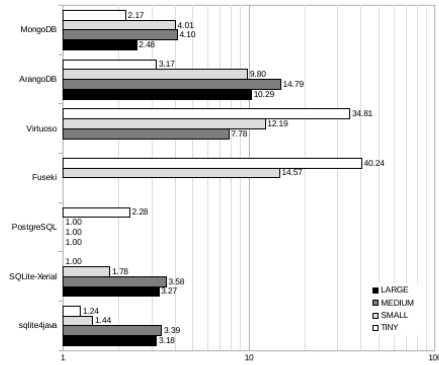
4 EXTRACTED FIGURES



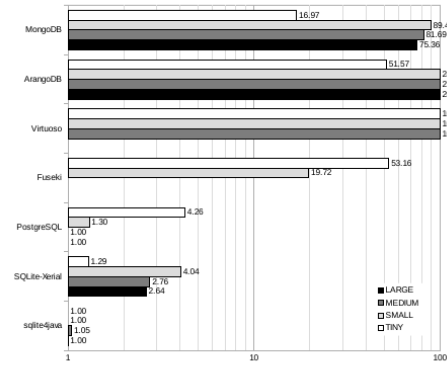
Evaluating Query and Storage Strategies for RDF Archives · 2019

<https://journals.sagepub.com/doi/10.3233/SW-180309>

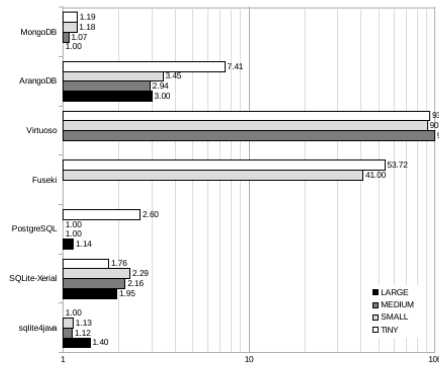
3 PAGE RENDERERS



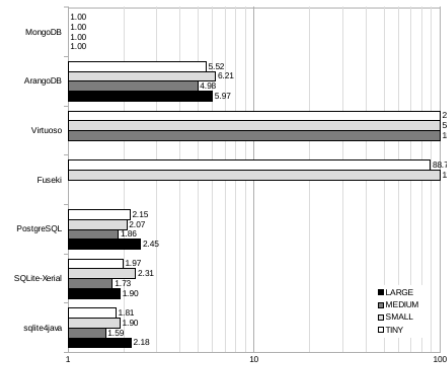
(a) ENTITY_RETRIEVAL_BY_ID_ONE_ENTITY



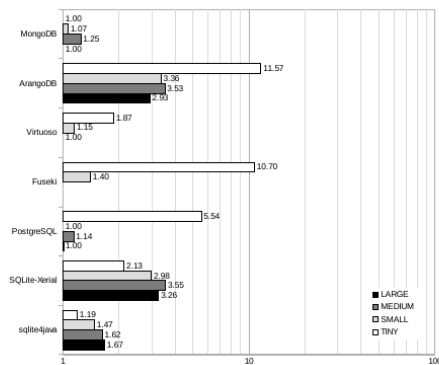
(b) ENTITY_RETRIEVAL_BY_ID_100_ENTITIES



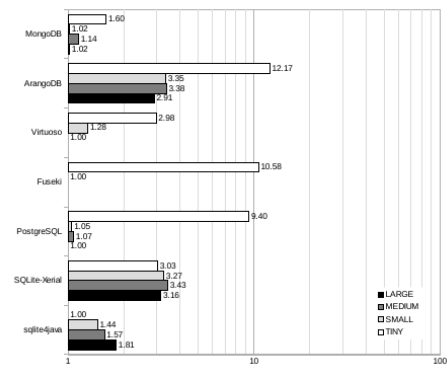
(c) CONDITIONAL_TABLE_SCAN_ALL_STUDIES



(d) CONDITIONAL_TABLE_SCAN_ALL_BIBLIOGRAPHIC_RESOURCES

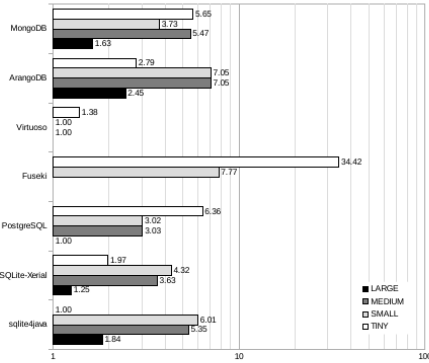


(e) AGGREGATE_ISSUES_PER_DECADE_TOP100

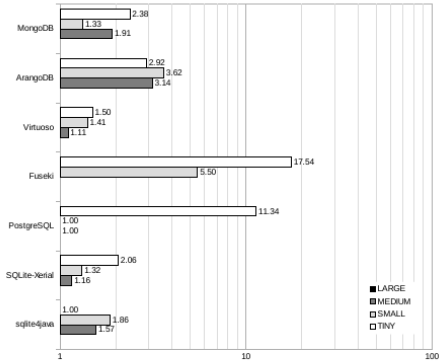


(f) AGGREGATE_ISSUES_PER_DECADE_ALL

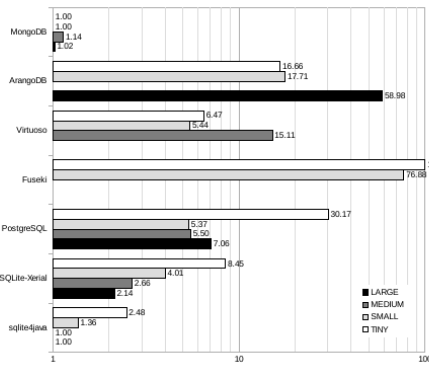
Figure 2. Query performances by scenario, database and test series, each normed by the best performer (1.0 is the best score archived).



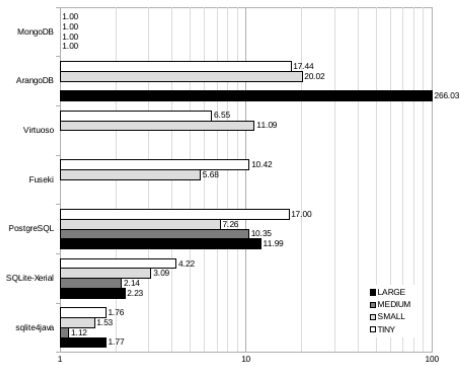
(a) AGGREGATE_PUBLICATIONS_PER_PUBLISHER_TOP10



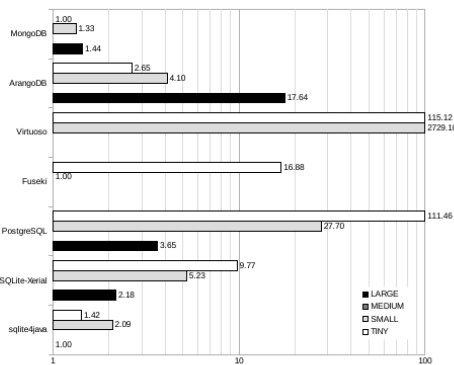
(b) AGGREGATE_PUBLICATIONS_PER_PUBLISHER_ALL



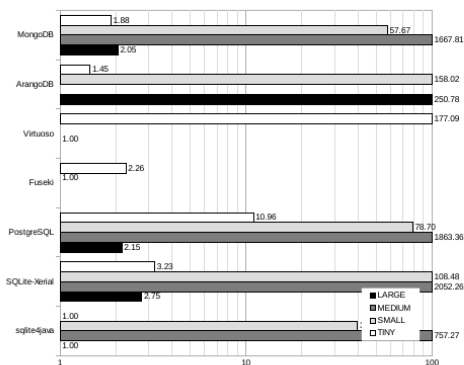
(c) UPDATE_HIGH_SELECTIVITY_NON_ISSUED



(d) UPDATE_LOW_SELECTIVITY_PAPER_MEDIUM



(e) DELETE_HIGH_SELECTIVITY_NON_ISSUED



(f) DELETE_LOW_SELECTIVITY_PAPER_MEDIUM

Figure 3. Query performances by scenario, database and test series, each normed by the best performer (1.0 is the best score archived).

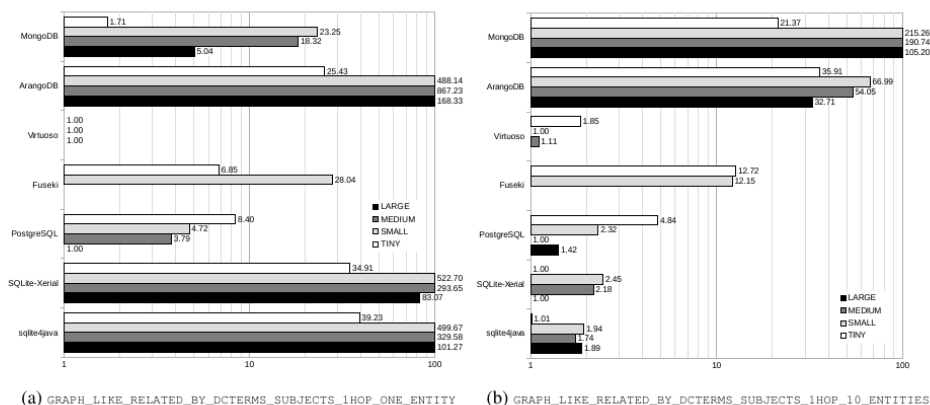


Figure 4. Query performances by scenario, database and test series, each normed by the best performer (1.0 is the best score archived).

4. Observations

The following observations are based on the conducted query performance comparison of sqlite4java, SQLite-Xerial, PostgreSQL, Apache Jena Fuseki, Virtuoso, MongoDB, and ArangoDB.

4.1. Stability of Operation

A first observation is the fact that errors occurred on all DBMSs except the relational approaches, reproducible for all kinds of dataset sizes.

At very low data sizes such as 100,000 entries, Virtuoso, ran into an error during query execution¹⁰. With growing data sizes, the number of execution problems increases as well. With 1,7 million records, Virtuoso¹¹ and ArangoDB¹² show occasional but repeatable query execution issues.

Despite the issue in MEDIUM, ArangoDB loads the largest test series LARGE, consisting of 5.7 million records, flawlessly. A single issue can be observed

for MongoDB¹³. Fuseki times out during load¹⁴. Syntax errors in the LARGE data source prevent Virtuoso from loading it entirely¹⁵.

4.2. Loading Times

With all approaches, loading of 1,000 (TINY) or 100,000 entries (SMALL) only requires a few milliseconds or seconds. With larger dataset sizes, Virtuoso in particular can make use of its inherent advantage of loading data dumps directly: it is twice as fast as the fastest non-native RDF store. However, there is a critical weakness of this loading method: even small syntax errors can invalidate the entire loading process. This happens in case of the LARGE test series: One of the two datasets involved has a syntax error, which causes Virtuoso to fail when loading it¹⁶. The other triple store, Fuseki, starts failing with MEDIUM data sizes because of timeouts¹⁴. Even though very large timeout thresholds are configured, they don't seem to be sufficient.

¹⁰ https://github.com/heussd/lodwhbench/blob/master/results/Errors_Virtuoso_SMALL.txt, last access on 2017-01-18.

¹¹ https://github.com/heussd/lodwhbench/blob/master/results/Errors_Virtuoso_MEDIUM.txt, last access on 2017-01-18.

¹² https://github.com/heussd/lodwhbench/blob/master/results/Errors_Arango_MEDIUM_Since_SCHEMA_CHANGE_MIGRATE_RDF_TYPE.txt, last access on 2017-01-18.

¹³ https://github.com/heussd/lodwhbench/blob/master/results/Errors_MongoDB_LARGE_Aggregation.txt, last access on 2017-01-18.

¹⁴ https://github.com/heussd/lodwhbench/blob/master/results/Errors_Fuseki_Load_of_LARGE_and_MEDIUM.txt, last access on 2017-01-18.

¹⁵ https://github.com/heussd/lodwhbench/blob/master/results/Errors_Virtuoso_LARGE.txt, last access on 2017-01-18.

¹⁶ https://github.com/heussd/lodwhbench/blob/master/results/Errors_Virtuoso_LARGE.txt, last access on 2017-01-18.

D_0	U_1	$D_1 = U_1(D_0)$
$G_0^0 = \{\langle :USA, a, :Country \rangle, \langle :Cuba, a, :Country \rangle, \langle :USA, :dr, :Cuba \rangle\}$	$\hat{u}_2 = \{\hat{u}_2^+ = \{G^2\}, \hat{u}_2^- = \emptyset\}$	$G_1^0 = \{\langle :USA, a, :Country \rangle, \langle :Cuba, a, :Country \rangle, \langle :France, a, :Country \rangle\}$
$G_0^1 = \{\langle x:JFK, a, x:Airport \rangle\}$	$u_1^0 = \{u_1^{0+} = \{\langle :France, a, :Country \rangle\}, u_1^{0-} = \{\langle :USA, :dr, :Cuba \rangle\}\}$	$G_1^1 = \{\langle x:JFK, a, x:Airport \rangle\}$
	$u_1^1 = \{\emptyset, \emptyset\}$	$G_1^2 = \emptyset$

Fig. 2. A dataset archive $\mathcal{A}$ with two revisions D_0, D_1 . The first revision contains two graphs, the default graph G^0 and G^1 . The dataset update U_1 (i) modifies G^0 , (ii) leaves G^1 untouched, and (iii) creates a new graph G^2 , all with local revision 1.

the revision's commit time $ts(\rho)$. This depends on the system's data model.

3. Framework for the Evolution of RDF Data

This section proposes *RDFev*, a framework to understand the evolution of RDF data. The framework consists of a set of metrics and a software tool to calculate those metrics throughout the history of the data. The metrics quantify the changes between two revisions of an RDF graph or dataset and can be categorized into two families: metrics for low-level changes, and metrics for high-level changes. Existing benchmarks, such as BEAR [19], focus on low-level changes, that is, additions and deletions of triples. This, however, may be of limited use to data maintainers, who may need to know the semantics of those changes, for instance, to understand whether additions are creating new entities or editing existing ones. On these grounds, we propose to quantify changes at the level of entities and object values, which we call high-level.

RDFev takes each version of an RDF dataset as an RDF dump in N-triples format (our implementation does not support multi-graph datasets and quads for the time being). The files must be provided in chronological order. *RDFev* then computes the different metrics for each consecutive pair of revisions. The tool is implemented in C++ and Python and uses the RocksDB³ key-value store as storage and indexing backend. All metrics are originally defined for RDF graphs in the state of the art [19], and have been ported to RDF datasets in this paper. *RDFev*'s source code is available at <https://relweb.cs.aau.dk/rdfev>.

³<http://rocksdb.org>

3.1. Low-level Changes

Low-level changes are changes at the triple level. Indicators for low-level changes focus on additions and deletions of triples and vocabulary elements. The vocabulary $\Upsilon(D) \subset \mathcal{I} \cup \mathcal{L} \cup \mathcal{B}$ of an RDF dataset D is the set of all the terms occurring in triples of the dataset. Tracking changes in the number of triples rather than in the raw size of the RDF dumps is more informative for data analytics, as the latter option is sensitive to the chosen serialization format. Moreover an increase in the vocabulary of a dataset can provide hints about the nature of the changes and the novelty of the data incorporated in a new revision. All metrics are defined by Fernández et al. [19] for pairs of revisions i, j with $j > i$.

Change ratio. The authors of BEAR [19] define the change ratio between two revisions i and j of an RDF graph G as

$$\delta_{i,j}(G) = \frac{|u_{i,j}^+| + |u_{i,j}^-|}{|G_i \cup G_j|}. \quad (1)$$

$\delta_{i,j}$ compares the size of the changes between two revisions w.r.t. the revisions' joint size. Large values for $\delta_{i,j}$ denote important changes in between the revisions. For a more fine-grained analysis, Fernández et al. [19] also proposes the insertion and deletion ratios:

$$\delta_{i,j}^+ = \frac{|u_{i,j}^+|}{|G_i|} \quad (2) \quad \delta_{i,j}^- = \frac{|u_{i,j}^-|}{|G_i|}. \quad (3)$$

We now adapt these metrics for RDF datasets. For this purpose, we define the size of a dataset D as $sz(D) = \sum_{G \in D} |G|$ and the size of a dataset change-set U as $sz(U) = sz(U^+) + sz(U^-)$ with $sz(U^+) = \sum_{u \in U} |u^+|$ and $sz(U^-) = \sum_{u \in U} |u^-|$. With these def-

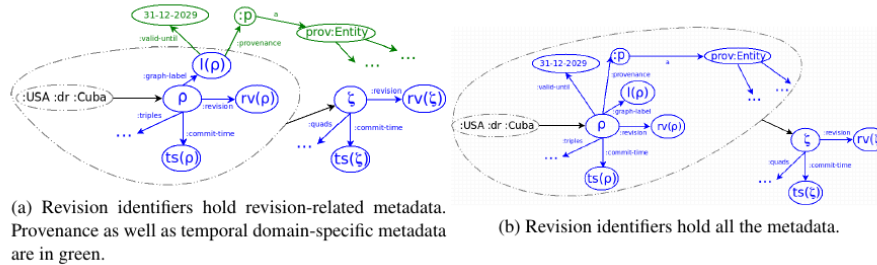


Fig. 7. Two logical models to handle metadata in RDF archives. The namespace *prov* corresponds to the PROV-O namespace.

(i) solutions with limited or no support for concurrent updates [3, 13, 24, 43, 52], (ii) solutions inspired by version control systems such as Git [4, 26, 31, 47, 54], and (iii) approaches with full support for highly concurrent updates [35]. Git-like solutions are particularly interesting for collaborative efforts such as DBpedia, because it is feasible to delegate users the task of conflict management. Conversely, fully automatically constructed KBs such as NELL [11] or data-intensive (e.g., streaming) applications may need the features of solutions such as x-RDF-3X [35]. Consequently, we propose a modular design that separates the concurrency layer from the storage backend. Such a middleware could take care of enforcing a consistency model for concurrent commits either automatically or via user-based conflict management. The layer could also manage the additional metadata for features such as branching and tagging. In that light, collaborative version control systems for RDF [4, 26, 47] become an application of fully-fledged RDF archiving.

Storage paradigm. RDF archiving differs from standard RDF management in an even more imperative need for storage efficiency. As shown by Taelman et al. [52], the existing storage paradigms shine at different types of queries. Hence, supporting arbitrary queries while being storage-efficient requires the best from the IC, CB, FB, and TB philosophies. A hybrid approach, however, will inevitably be more complex and will introduce further parameters and trade-offs. For example, CB solutions must define a policy to decide when to store a revision as a snapshot or as a delta, trading disk space for lower response time for VM queries. Likewise, TB solutions must build further indexes to efficiently answer V queries. Since CV and CD queries require further query planning, their performance depends on an efficient support for VM and DM queries and the capabilities of the query engine.

Depending on the host available main memory, a robust archiving system could materialize the latest revision of a dataset (or parts of it) in memory as done by some solutions [4].

Accounting for evolution patterns. As our study in Section 4 shows, the evolution patterns of RDF archives can change throughout time leading even, to decreases in dataset size. With that in mind, we envision an adaptive data-oriented system that relies on the metrics proposed in Section 3 and adjusts its parameters according to the archive's evolution. For example, we saw in Section 6 that a large number of deletions can negatively impact Ostrich's query runtime, hence, such an event could trigger the construction of a complete snapshot of the dataset in order to speed-up VM queries. In the same spirit and assuming some sort of dictionary encoding, an increase in the vocabulary dynamicity could increase the number of bits used to encode the identifiers of RDF terms in the dictionary. Those changes could be automatically carried out by the archiving engine, but could also be manually set up by the system administrator after an analysis with RDFev. A design philosophy that we envision to explore divides the history of each graph in the dataset in intervals such that each interval is associated to a block file. This file contains a full snapshot plus all the changesets in the interval. It follows that the application of a new changeset may update the latest block file or create a new one. This action could be automatically executed by the engine or triggered by the system administrator. For instance, if the archive is the back-end of a version control system, new branches may always trigger the creation of snapshots. This base architecture should be enhanced with additional indexes to speed up V queries and adapted compression for the dictionary and the triples. Finally as we expect long dataset histories, it is vital for solutions to improve

Angle 5 — Composite & Assembly Versioning

Towards a Unified Framework for Version Modeling · 1988

<https://dl.acm.org/doi/10.1145/98163.98172>

⚠ Scanned tech report with no text layer; figure pages could not be auto-isolated. (Version-graph and configuration-tree figures exist in the source.)

Versions of Schema for Object-Oriented Databases · 1988

<https://www.vldb.org/conf/1988/P148.PDF>

2 PAGE RENDERS

versions are shown with the objects that are visible to them. Under the initial schema version SV-0, two versioned objects, a1 and a2 are created, along with their first version instances a1.v0 and a2.v0, and then a second version instance of a1, a1.v1, is derived. The objects a1 and a2 belong to the same class, and has three attributes, at1, at2, and at3. Then a new schema version, SV-1 is derived from SV-0, by deleting attribute at3 from the class. All objects created under SV-0 are now visible to SV-1, without the deleted attribute at3. Further, a new version instance v1 of a2 is created, and a new versioned object a3 (along with its first version instance a3.v0) is created under SV-1. Then a new schema version SV-2 is derived from SV-1 by adding a new attribute at4 to the class. All objects and their attributes visible to SV-1 are now made visible to SV-2, along with the new attribute at4. A new version instance of a3 is derived under SV-2.

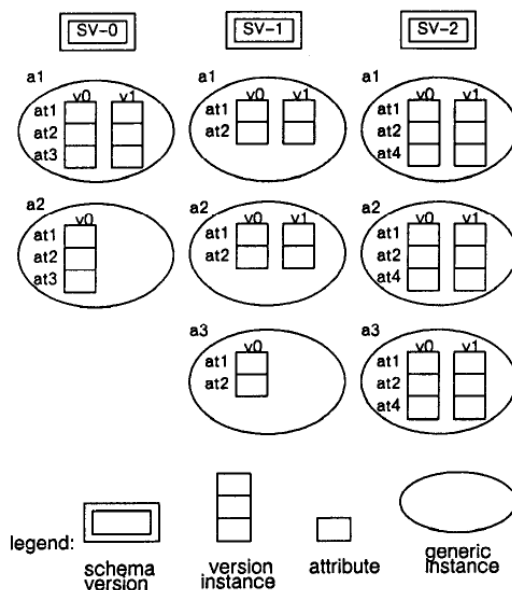


Figure 1. Example Schema Versions

Now, we consider the direct-access-scope update rule. There may be conflicts between different schema versions derived from the same schema version SV with respect to updatability of the access scope of SV. For example, a schema version SV-j may have been derived from SV by leaving the access scope of SV updatable under SV, and a new schema version SV-k is then derived from SV by making the access scope of SV non-updatable under SV. We take the view that when the user derives a schema version SV-j from SV by leaving the access scope of SV updatable under SV, the user in ef-

fect does not care whether the access scope of SV is updatable (which means any changes will automatically propagate to SV-j). Therefore, we give preference to the default access-scope inheritance rule over exceptions. In the current example, derivation of the new schema version SV-k will make the access scope of SV non-updatable under SV. By the same token, if SV-k was derived first, the access scope of SV will remain non-updatable under SV, when SV-j is derived next.

Next, we consider the inherited-access-scope update rule. If objects are inherited into SV-j from SV-i, the effects of the updates and deletes made under SV-j are visible only to SV-j; that is, when viewed from SV-i, it is as if the updates and deletes had never taken place. We restrict insertion (creation) of new objects to only the current schema version; that is, newly created objects belong to the direct access scope of the schema version under which they are created.

In the case of an update (replace) of an inherited object under a schema version SV, the new object is made visible to all descendant schema versions of SV which inherit the access scope of SV. The new object persists, even if the old object is deleted under its creator schema version.

Further, when an inherited object is deleted under a schema version SV, the object will not be visible to SV and any descendant schema version of SV which inherits the access scope of SV. However, the object will continue to be accessible to any other schema version which includes the creator schema version SV in its access scope.

5. IMPLEMENTATION ISSUES

In this section, we discuss auxiliary data structures for objects to efficiently support versioning of the schema, detailed algorithms for accessing objects, and storage representation for the schema versions.

5.1 AUXILIARY DATA STRUCTURES FOR OBJECTS

To support object manipulation in the presence of versions of schema, we need to include in every object one system-defined attribute, called the creator attribute, and to associate a data structure, called an anchor instance, with every object. The *creator attribute* of an object indicates the creator schema version of the object. The *anchor instance* of an object is a data structure which describes a set of copies of the object; recall that, in a similar manner, a generic instance of a versioned object describes the set of version instances of the object. Algorithms for object fetch, insert, delete, and replace are presented in the next subsection.

To support delete or replace of an inherited object, the system will create a new copy of the object when the object is first deleted or replaced under a schema version

which is not its creator schema version. An object, when first created, exists without an anchor instance; however, it will carry the identifier of the creator schema version. An anchor instance, and each of the copies of the object it describes, are all identified by the same object identifier of the object; this is necessary so as not to invalidate existing references to the object. The creator attribute in each copy of the object is used to distinguish one copy from any other.

The anchor instance of an object consists of the following system-defined attributes.

1. a list of terminator schema versions
2. a list of copies of the object

A *terminator schema version* is the schema version under which the object was deleted. We note that terminator schema version can only be a descendant of the creator schema version; if an object is deleted or replaced under its creator schema version, it is physically deleted or replaced, respectively.

Figure 2 illustrates the way in which an anchor instance is created and manipulated. In the figure, the anchor instance is shown with a terminator, and each copy of the object includes the creator schema version. The anchor instance and copies of the object are created in the following sequence. The object initially exists as copy-0, and SV-0 is its creator schema version. The anchor instance is created when the object is updated under schema version SV-4, and a new copy of the object, copy-1, is created with SV-4 as the creator schema version. The anchor instance describes two copies of the object. Then copy-0, which is still accessible to schema version SV-1, is deleted under SV-1, causing schema version SV-1 to be recorded in the terminator attribute of the anchor instance.

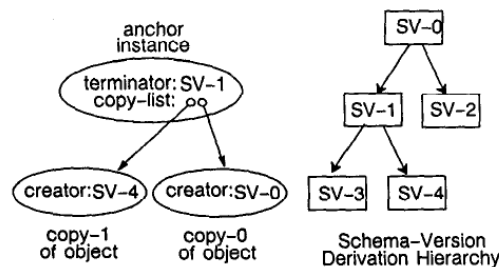


Figure 2. An Anchor Instance and Object Copies

5.2 OBJECT ACCESS ALGORITHMS

(This section may be skipped without loss of continuity.) The following algorithms precisely describe the way in which an object is fetched, inserted, replaced, and deleted. They use the augmented object structure and the

anchor instance discussed earlier. In the algorithms, the parameter *sv* denotes the current schema version. We assume the existence of two boolean functions: (1) *Ancestor-of*, which returns TRUE when its first argument is an ancestor schema version whose access scope is inherited by the schema version specified in the second argument; and (2) *Frozen*, which returns True when the argument specifies a schema version whose direct access scope is non-updatable under the schema version. For convenience, we define another boolean function

Ancestor-Of* (sv1, sv2) =
Ancestor-Of (sv1, sv2) OR sv1 = sv2.

```

find_closest_copy (copy-list, sv)
/* routine for finding the ancestor copy that is closest to sv */
for each c in copy-list
  if ancestor-of* (c.creator, sv) return c;
/* an ancestor copy found */
end-for;
return nil;
end find_closest_copy;
  
```

```

copy_blocked (terminators, sv, copy-sv)
/* routine for checking if a copy is visible under sv */
for each t in terminators
  if (ancestor-of* (t, sv)) and (ancestor-of (copy-sv, t))
    return TRUE; /* copy-sv is blocked from sv by t */
end-for;
return FALSE; /* sv is not blocked by any terminator */
end copy_blocked;
  
```

```

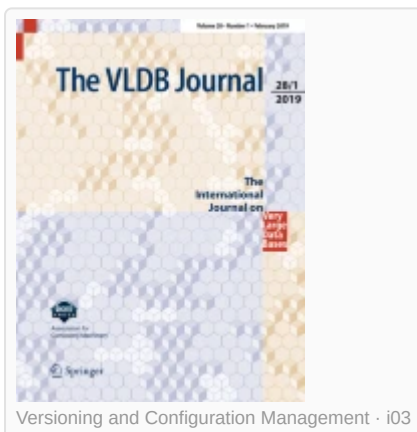
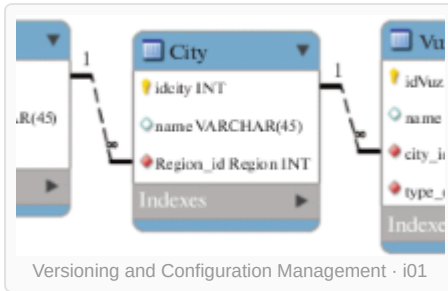
ALGORITHM_SV_FETCH (object-id, sv)
object <- locate_object (object-id);
if (object is an anchor instance)
  do anchor-instance <- object;
  object <- find_closest_copy
    (anchor-instance.copy-list, sv);
  if (object = nil) or
    copy_blocked(anchor-instance.terminators,
      sv, object.creator)
    error; /* no copy is visible under sv */
  end-do;
else if not (ancestor-of* (object.creator, sv))
  error; /* the only existing copy is not visible */
return object;
end algorithm_sv_fetch;
  
```

```

ALGORITHM_SV_INSERT (object, sv)
if frozen (sv) error;
object.creator <- sv;
allocate an id for the object and return the id;
end algorithm_sv_insert;
  
```

```

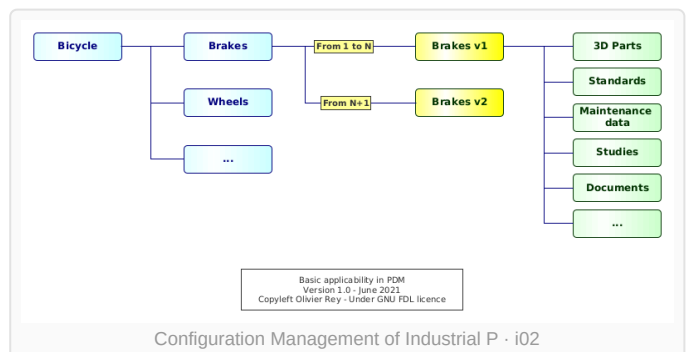
ALGORITHM_SV_DELETE (object-id, sv)
if frozen (sv) error;
object <- locate_object (object-id);
if (object is an anchor instance)
  
```

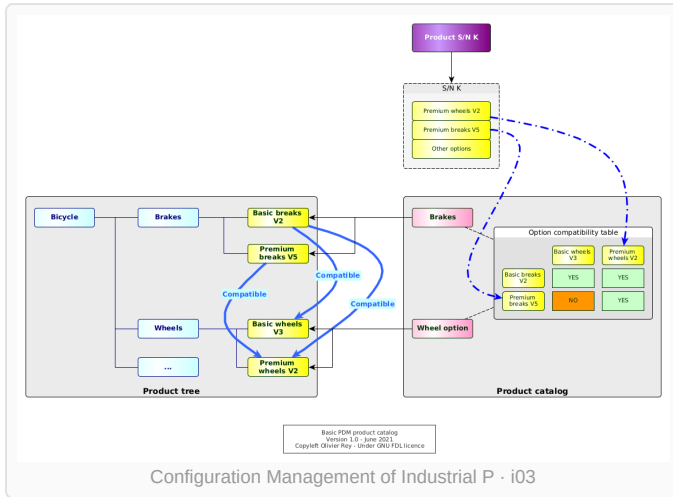


Configuration Management of Industrial Products in PDM/PLM · 2021

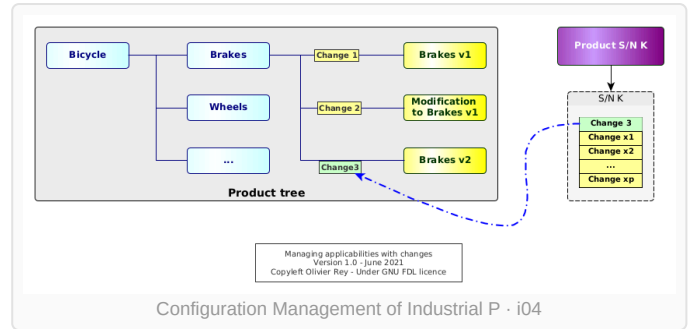
<https://orey.github.io/papers/articles/conf-mgt/>

7 EXTRACTED FIGURES

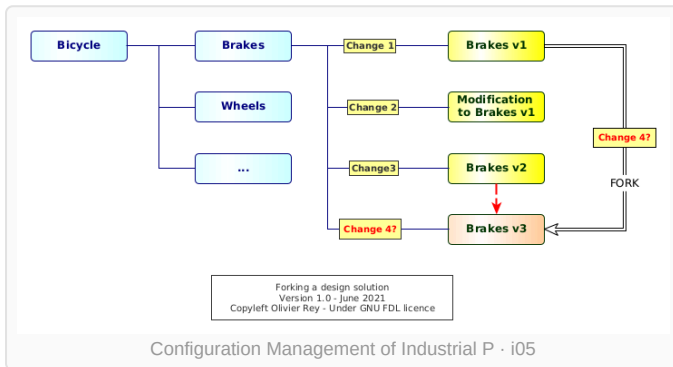




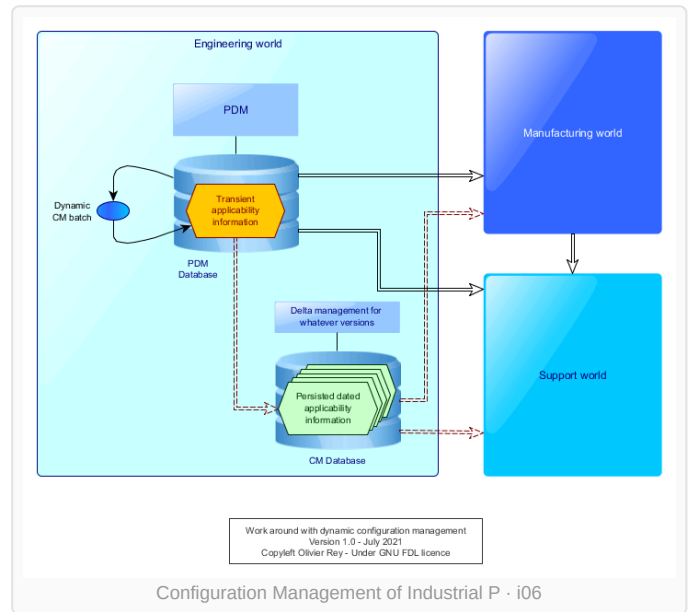
Configuration Management of Industrial P - i03



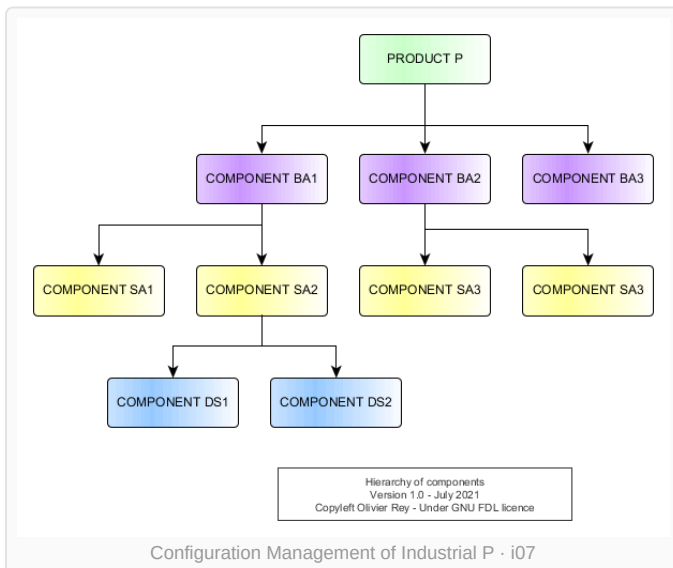
Configuration Management of Industrial P - i04



Configuration Management of Industrial P - i05



Configuration Management of Industrial P - i06



Configuration Management of Industrial P - i07